Séance 1 — Le Web et le HTML

À la fin de cette séance, vous comprendrez ce qui se passe entre le moment où vous tapez une adresse et celui où la page s'affiche, et vous serez prêt à installer votre environnement et à écrire votre première page web dès la séance 2.

Faisons connaissance

Avant de commencer, un tour de table :

- **Quel est votre prénom et nom ?**
- **Pourquoi avez-vous choisi la section informatique de gestion ?**
- **Avez-vous suivi d'autres études ou formations auparavant ?**
- **Qu'attendez-vous de ce cours de web ?**
- **Avez-vous déjà une expérience avec la création ou l'utilisation de sites web (HTML, CSS, etc.) ?**

Qui suis-je ?

Parcours académique

- 2015 — Bachelier en sciences mathématiques, ULB
- 2017 — Master en sciences actuarielles, ULB
- 2023 — Bachelier en informatique de gestion, EAFC Evere
- 2024 — Executive Master in Digital & IT Essentials, VUB/ULB

Parcours professionnel

- 2015-2016 — Professeure de mathématiques, enseignement secondaire
- 2017 — Stagiaire en assurance vie, AG Insurance
- 2017-2018 — Consultante, EY
- 2018-2020 — Interne actuaire non-vie, Generali, Athora et Bâloise
- 2020-2022 — Actuaire pricing, Touring Assurances
- 2022-2024 — Actuaire performance monitoring, Belfius Assurances
- 2025 – … — Manager Operational Performance & Technical Support Non-Life, Belfius Assurances

- 🎓 2025 – … — Professeure, EAFC Evere

Plan du cours

Le programme complet des 10 séances est disponible sur la page Vue d'ensemble.

Grille d'évaluation

Pour atteindre le seuil de réussite, vous devrez prouver que vous êtes capable de réaliser des pages conformes, face à une structure informatique opérationnelle connectée à Internet, en utilisant le vocabulaire technique et l'orthographe adéquats, et en respectant les normes en vigueur.

Nom & prénom du chargé de cours : MARIN PONCE Rubi
Nom & prénom de l'étudiant :
Section : bachelier en informatique de gestion
Unité d'enseignement : WEB : Principes de base
Code de l'UF : 7534 29 U32 D1
Implantation : Evere

EAFC Evere

EAFC EVERE
Enseignement pour Adultes
et Formation Continue
Avenue Constant Permeke 4
1140 Bruxelles
☏ 02 701 97 97
www.eafcevere.eu

WALLONIE-BRUXELLES ENSEIGNEMENT

PÔLE ACADÉMIQUE DE BRUXELLES

POUR ATTEINDRE LE SEUIL DE REUSSITE, L'ETUDIANT DEVRA PROUVER QU'IL EST CAPABLE DE :

Face à une structure informatique opérationnelle connectée à Internet, disposant des logiciels appropriés et de la documentation nécessaire, en utilisant le vocabulaire technique et l'orthographe adéquate, et en respectant les normes et standards en vigueur, sur base d'un cahier des charges technique proposé par le chargé de cours, comprenant au moins l'intégration des feuilles de styles et le respect de la sémantique,

Capacité(s) terminale(s) ou C.T/AA	Critères observables	Niveaux de performance : ACQUIS	NON ACQUIS
C.T/AA n°1 de réaliser des pages statiques, compatibles avec au minimum un navigateur récent du marché ;	Doctype correcte		
	Balises et attributs corrects		
	Liens (css et js compris) référencés sont corrects		
	Pages sont vaildes (validateur w3c)		
Bilan C.T/AA	**Conditions de réussite de la C.T/AA (3 critères, le dernier obligatoire)**		
C.T/AA n°2 de transférer et de tester ces pages statiques sur un serveur.	Utiliser un client ftp		
	Récupérer le code d'un serveur via client ftp		
	Transférer le code sur un serveur via client ftp		
Bilan C.T/AA	**Conditions de réussite de la C.T. (3 critères)**		
Seuil de réussite	**Toutes les C.T./A.A.**	**50%**	

Si le seuil de réussite est atteint, il sera tenu compte des critères suivants pour la détermination du degré de maîtrise

N° des degrés de maîtrise		Niveau de maîtrise	Sous-total
N°1 utilisation pertinente des technologies appropriées,	Balises et attributs HTML appropriés	**10**	
	CSS, choix des sélecteurs et attributs	**5**	
N°2 la rigueur et le respect des spécificités du langage,	Syntaxe HTML	**5**	
	Syntaxe CSS	**5**	
N°3 le niveau de fiabilité des tests,	Validateur w3c	**10**	
N°4 le degré d'autonomie atteint,	Travaux à distance et participation en classe	**10**	
N°5 l'utilisation judicieuse du vocabulaire informatique.	Commentaires et participation en classe	**5**	
Bilan des 5 degrés de maîtrise :	**On additionne les différents résultats obtenus dans les bilans de maîtrise ci-dessus et on remet ce bilan sur un total de 50%**	**50**	
RESULTAT FINAL DE L'UNITE D'ENSEIGNEMENT	**On additionne le bilan des C.T. et le bilan du degré de maîtrise**		**100**

SANCTION DE L'UNITE D'ENSEIGNEMENT (barrer les mentions inutiles) — **Réussite** / ~~**Ajournement**~~ / ~~**Refus**~~

Consignes pour la seconde session :

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Date :

Signature de l'étudiant :

Signature du professeur et/ou du chef d'atelier :

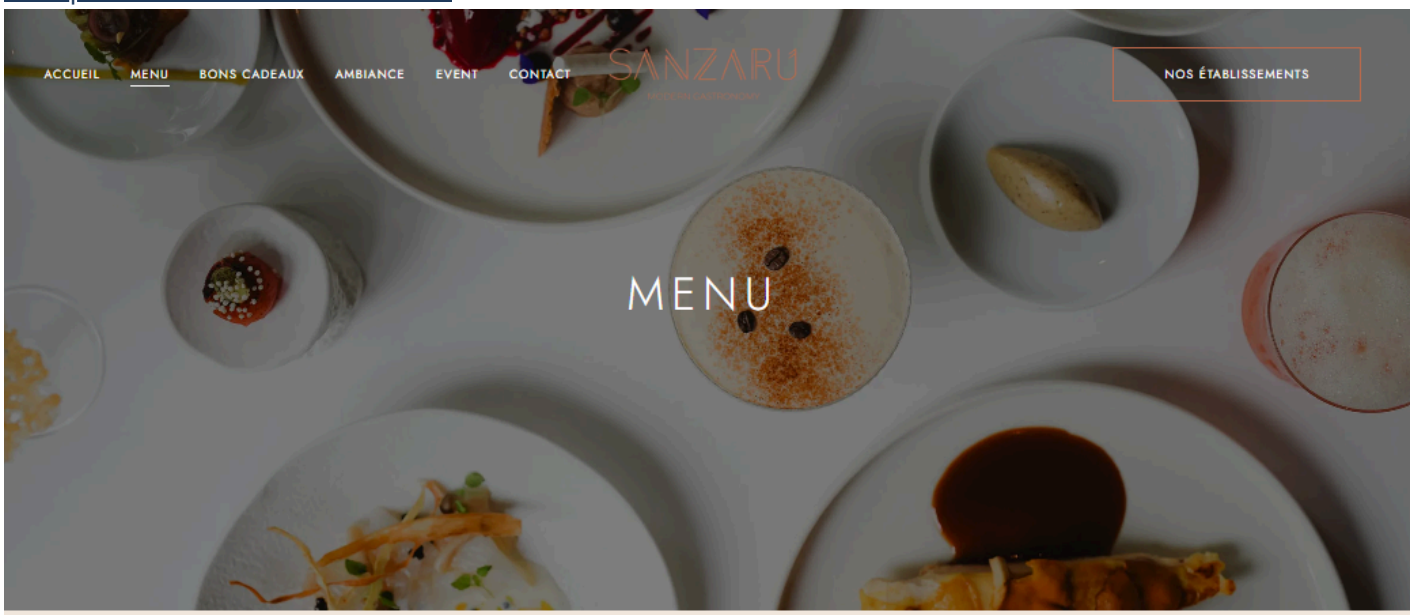
Sommaire

1. 1. Comprendre ce qu'est le Web
2. 2. Comment une page arrive à l'écran
3. 3. Le HTML
4. 4. Lire et comprendre une page HTML
5. 5. Les types de sites web : usages et fonctionnement
6. 6. Normes et validation
7. 7. La sécurité de base
8. 8. Exercices

1. Comprendre ce qu'est le Web

Imaginez le site d'un restaurant : une page d'accueil avec une photo, un menu, les horaires, un formulaire de réservation. Pour comprendre comment ce site existe et comment vous y accédez, il faut distinguer plusieurs notions : Internet, le Web, et le site web lui-même.

[Exemple de site web d'un restaurant]



L'EXPÉRIENCE SANZARU

LAISSEZ-VOUS GUIDER

Au rythme des saisons et des plus beaux arrivages, notre chef imagine des menus en constante évolution.

Leur composition reste volontairement secrète jusqu'à votre venue pour laisser toute sa place à la découverte et à l'émotion.

◊ DÉJEUNER ◊

MENU LUNCH - 55€

Décliné en 3 services

Accord vins 25€
Accord sans alcool 20€

MENU SANZA 65€

Décliné en 4 services

Accord vins 45€
Accord sans alcool 35€

◊ DÎNER ◊

MENU MIZA 85€

Décliné en 4 services

Accord vins 50€
Accord sans alcool 40€

MENU UMAMI 115€

Décliné en 6 services

Accord vins 70€
Accord sans alcool 55€

ALLERGIES & INTOLÉRANCES

Afin de vous accueillir dans les meilleures conditions, nous vous invitons à nous signaler toute allergie, intolérance ou contrainte alimentaire au moment de la réservation. Notre chef adaptera le menu en conséquence.

PRÊT À VOUS LAISSER SURPRENDRE ?

RÉSERVER UNE TABLE

CONTACTEZ-NOUS

T. +32 2 773 00 80
M. info@sanzaru.be

ADRESSE

Av. de Tervueren 292,
1150 Woluwe-Saint-Pierre

HEURES D'OUVERTURE

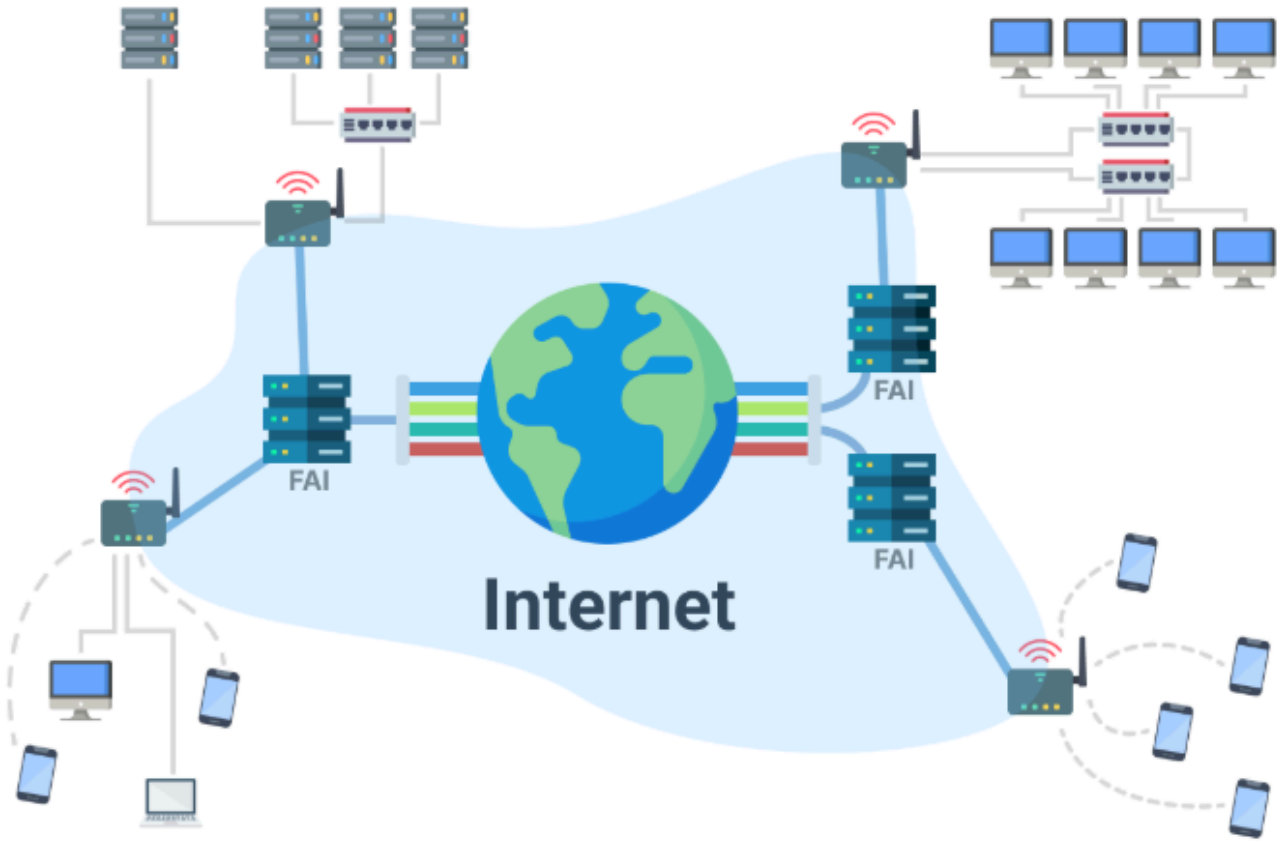
Déjeuner
Du mardi au vendredi
de 12h à 13h30
Dîner
Du mardi au samedi
de 19h à 20h30
Jours de fermeture
Dimanche et lundi

Internet

Internet est une infrastructure : un immense réseau mondial qui connecte entre eux des millions d'autres réseaux, par l'intermédiaire de **FAI** (Fournisseurs d'Accès à Internet).

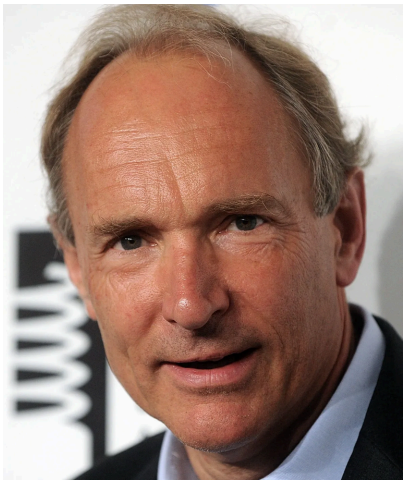
Chacun de ces réseaux relie toutes sortes d'équipements — ordinateurs personnels, serveurs, objets connectés, routeurs — grâce à une infrastructure physique (câbles, fibre optique, lignes téléphoniques) et à des protocoles standardisés comme TCP/IP, vus en *Bases des réseaux*.

Internet est donc le support qui permet à des machines, où qu'elles soient dans le monde, de s'échanger des données. Plusieurs services circulent sur ce support : le courrier électronique, le partage de fichiers, la visioconférence, et bien sûr le Web.



Le Web

Le **World Wide Web** (ou simplement « web ») est l'un de ces services qui fonctionnent sur Internet. Créé par **Tim Berners-Lee** en 1989 au CERN, c'est une collection massive de documents et de médias (images, vidéos, fichiers…) connectés entre eux comme une toile, grâce à des liens *hypertextes*.



Le Web repose sur des technologies précises, que vous allez utiliser tout au long de ce cours :

- **HTTP** : le protocole qui permet de demander et de recevoir des pages web.
- **HTML, CSS, JavaScript** : les langages utilisés pour structurer, styliser et animer ces pages.
- **URL** : l'adresse unique qui permet d'identifier et d'accéder à chaque page.

En résumé, le Web est comme un livre gigantesque, dont les pages seraient éparpillées partout sur la planète, et reliées entre elles par des liens.

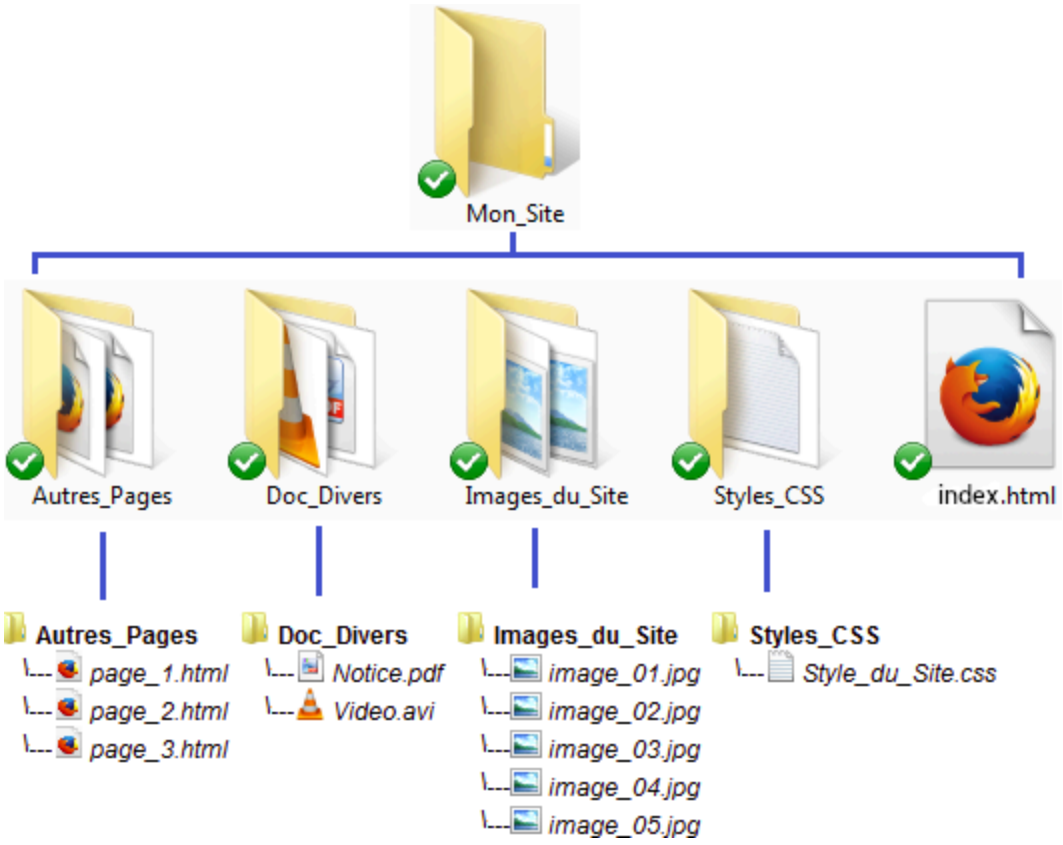
Ces 3 notions qui seront vues plus en détail ci-dessous.

Quelques repères de l'histoire du Web

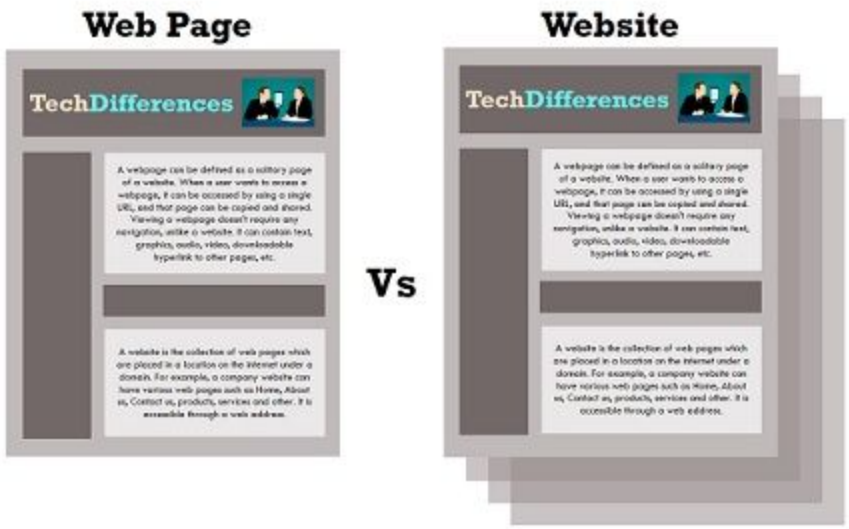
Année	Événement
1969	ARPANET, l'ancêtre d'Internet, relie ses quatre premiers ordinateurs aux États-Unis.
1983	Le protocole TCP/IP devient le standard d'Internet.
1989	Tim Berners-Lee propose le World Wide Web au CERN.
1990	Tim Berners-Lee crée le HTML, le protocole HTTP et le premier navigateur.
1991	Mise en ligne de la première page web de l'histoire.
1993	Mosaic, premier navigateur grand public à afficher des images.
1994	Netscape Navigator démocratise la navigation ; création du W3C.
1995	HTML 2.0, première version officiellement standardisée du HTML ; naissance de JavaScript.
1996	Le CSS fait son apparition.
1999	HTML 4.01, la version de référence pendant plus de dix ans.
2014	Publication du standard HTML5.
2019	Le W3C et le WHATWG s'accordent : le HTML devient un «standard vivant», mis à jour en continu.

Le site web

Un **site web** est un ensemble organisé de pages reliées entre elles et accessibles grâce à une adresse unique appelée URL. Ces pages, conçues pour s'afficher dans des navigateurs comme Chrome, Firefox, ..., peuvent contenir du texte, des images, des vidéos, des liens ou encore des fichiers à télécharger. L'ensemble est hébergé sur un serveur connecté à Internet. Lorsqu'un internaute saisit l'adresse dans son navigateur, celui-ci communique avec le serveur pour afficher les pages demandées.



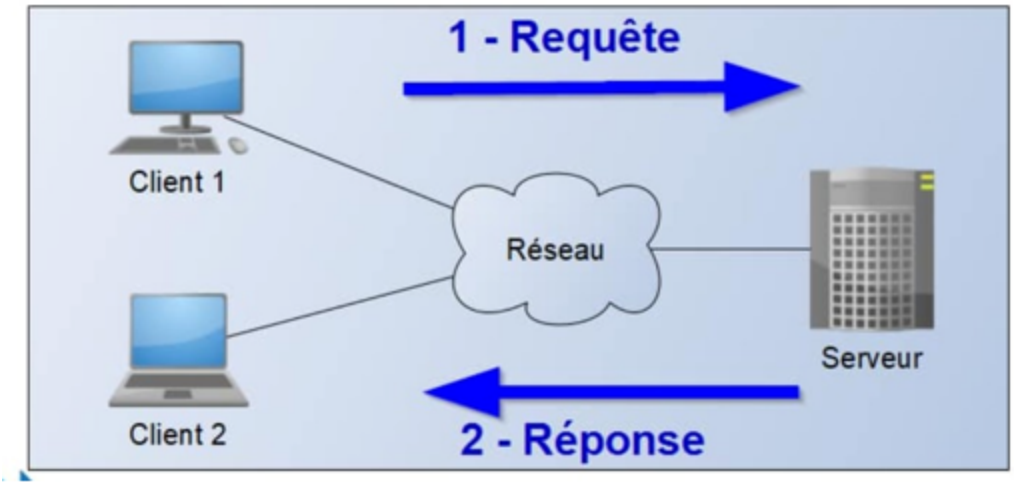
Attention à ne pas confondre avec une simple **page web** : un document unique, affiché dans le navigateur, avec sa propre URL — la page d'accueil, une page « Contact » ou « À propos » sont chacune une page web. Un site web est l'ensemble de ces pages, reliées entre elles et regroupées sous un même nom de domaine : une page d'accueil, des sous-pages, éventuellement un blog ou une boutique en ligne.



2. Comment une page arrive à l'écran

Le modèle client / serveur

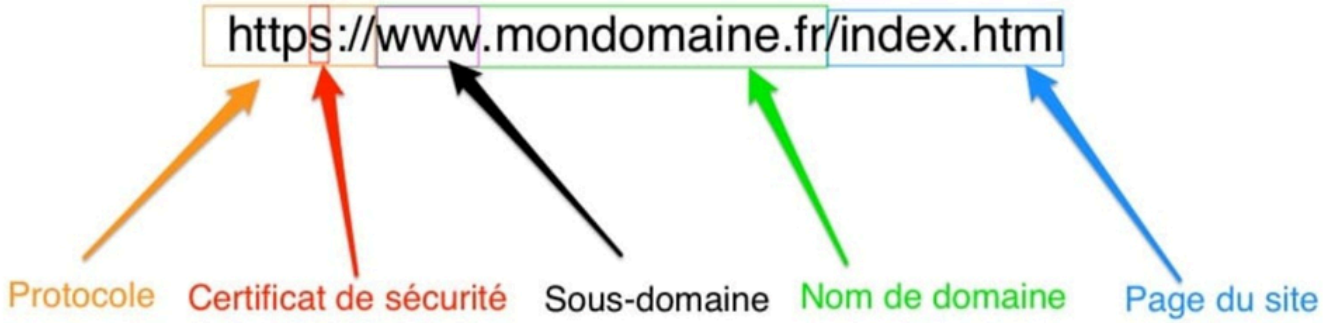
Votre navigateur est le **client**. Il envoie une *requête*. Une machine distante, le **serveur**, renvoie une *réponse* : le plus souvent, un fichier HTML.



> ### Rappel du cours de réseaux
>
> Vous avez vu en *Bases des réseaux* comment les données circulent (TCP/IP, adressage, DNS). Ici, on s'intéresse uniquement à la couche du dessus : ce que le Web ajoute par-dessus le réseau.

L'URL

Une adresse web se décompose toujours de la même façon :



Un nom de domaine et sous-domaines

Un **nom de domaine** est un nom lisible et facile à mémoriser qui permet d'accéder à un site ou à un service sur Internet. Il évite de devoir retenir une adresse IP numérique.

Le nom de domaine se compose notamment d'un nom (`mondomaine`) et d'une extension (`.fr`), appelée aussi domaine de premier niveau.

Un **sous-domaine** sert à créer une partie distincte d'un site web, tout en restant associé au même nom de domaine principal.

Exemple

Prenons le domaine `exemple.be` :

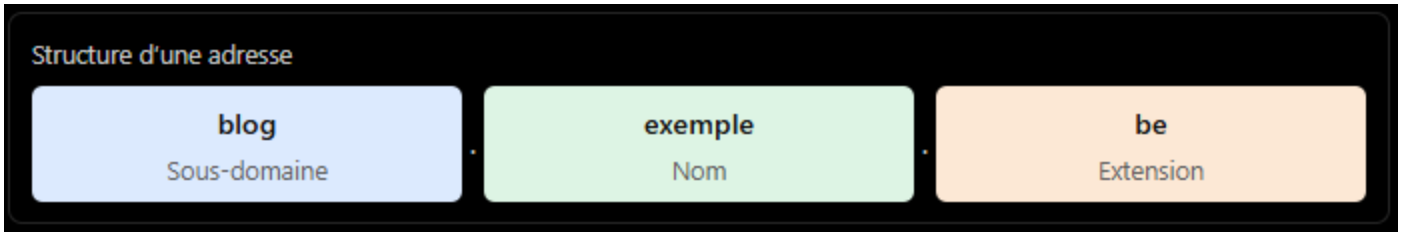
Exemples d'adresses et d'utilités possibles

Adresse	Utilité possible
`www.exemple.be`	Site web principal
`blog.exemple.be`	Blog de l'entreprise
`boutique.exemple.be`	Boutique en ligne
`support.exemple.be`	Assistance aux clients

Dans `blog.exemple.be` :

- `blog` est le sous-domaine
- `exemple.be` est le nom de domaine

Structure d'une adresse



Pourquoi utiliser un sous-domaine ?

- Organiser un site en plusieurs parties.

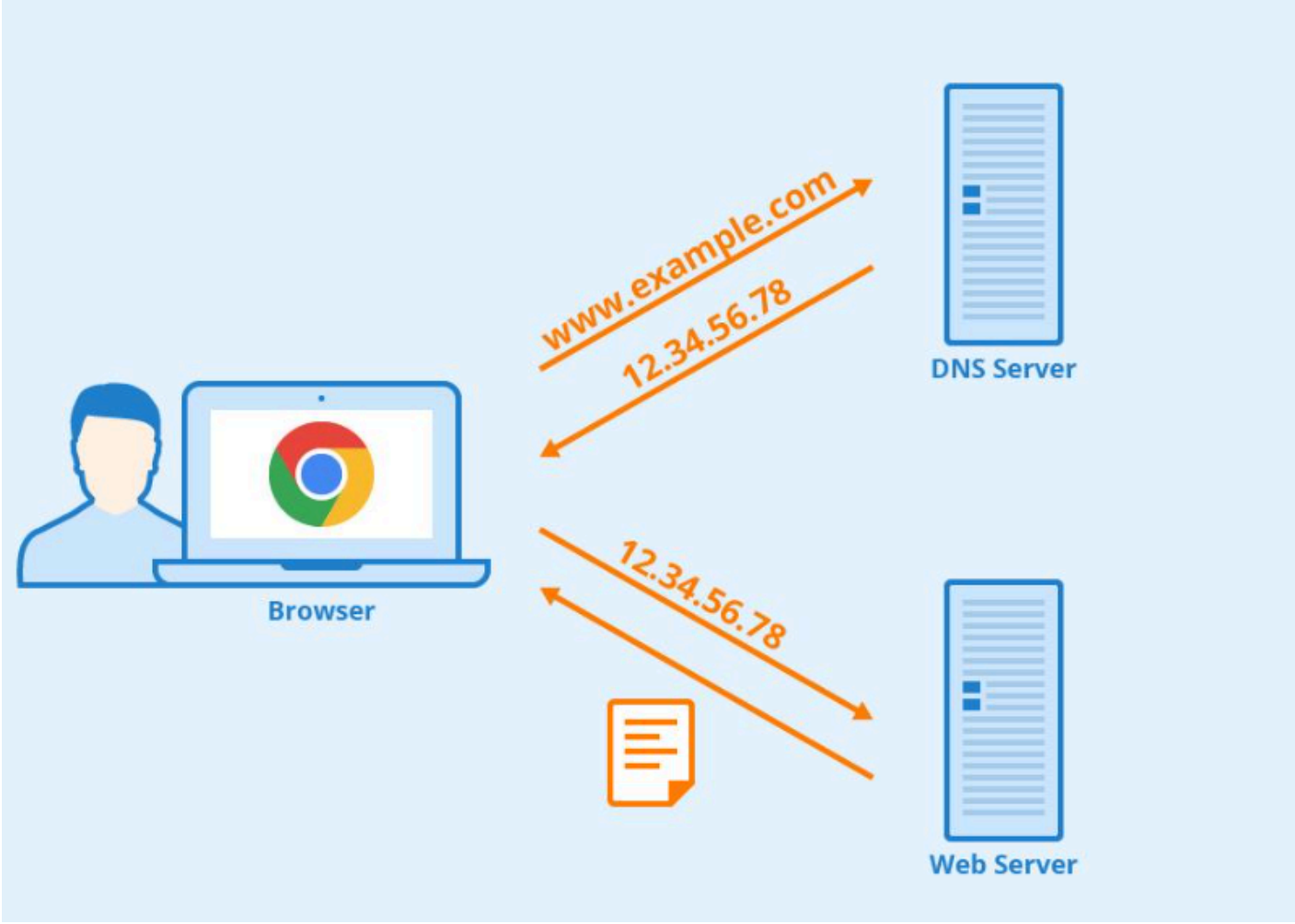
- Séparer des services : blog, boutique, espace client, etc.
- Configurer chaque partie différemment, éventuellement sur des serveurs distincts.

> ### À retenir
>
> Un sous-domaine n'est pas simplement un dossier. Par exemple, `exemple.be/blog` est un chemin dans le site, tandis que `blog.exemple.be` est un sous-domaine.
>
> - Autrement dit, `exemple.be` peut être considérer comme bâtiment et `/blog` comme une pièce à l'intérieur de ce bâtiment. Le blog fait partie du site principal. Il peut correspondre à une page, à une section ou à un ensemble de pages.
> - tandis que `blog.exemple.be` est un bâtiment distinct. Il peut être un site distinct, avec sa propre structure et ses propres pages.

Le DNS : l'annuaire du Web

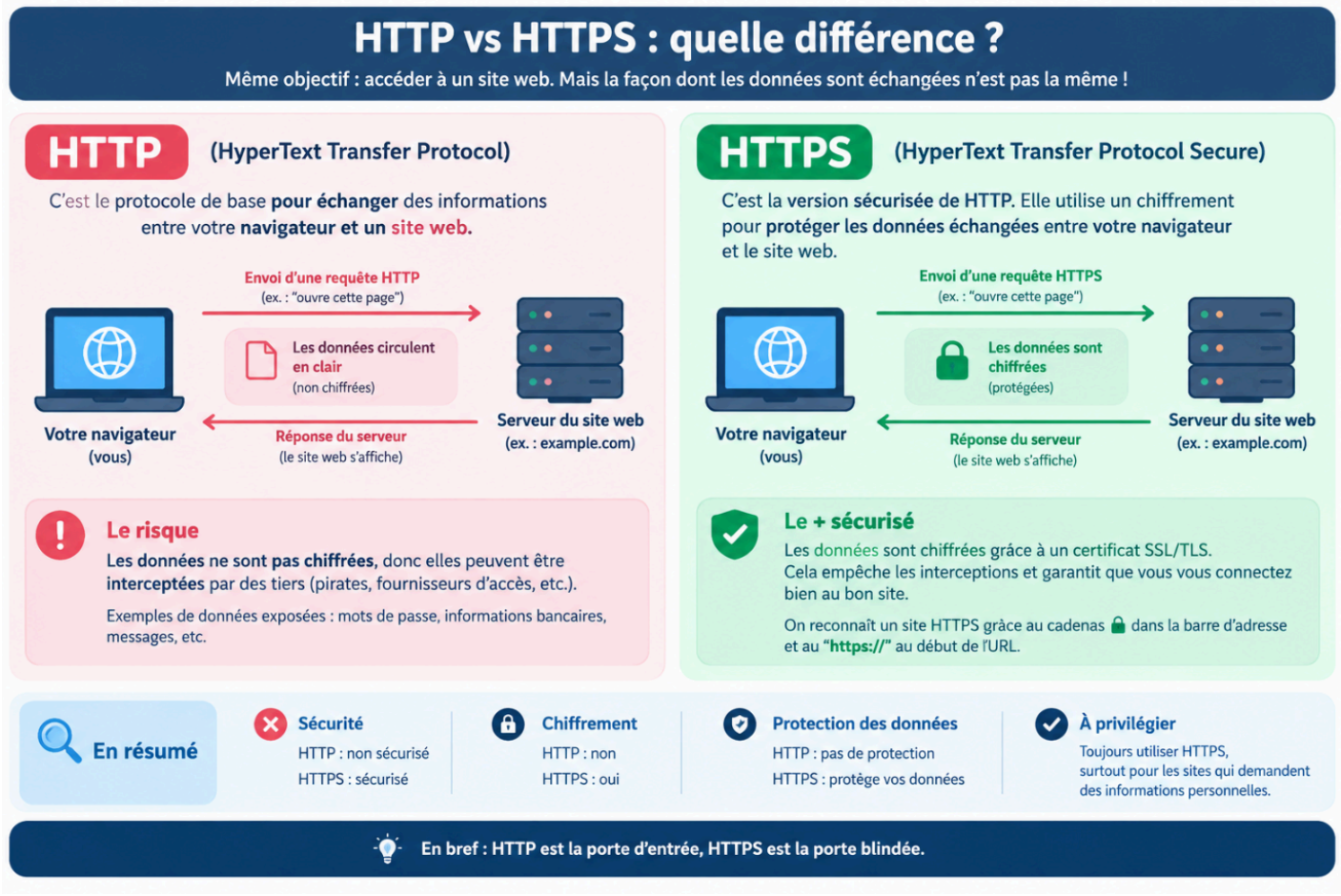
Sur Internet, une machine se trouve grâce à son **adresse IP**, une suite de nombres comme `12.34.56.78`. Les humains, eux, retiennent plus facilement des noms comme `www.example.com`.

Le **DNS** (*Domain Name System*) fait la traduction entre les deux, comme un annuaire téléphonique : on cherche un nom, on obtient un numéro. C'est la toute première étape de chaque visite d'un site. Nous y reviendrons à la séance 9, au moment de mettre vos pages en ligne sous votre propre adresse.



HTTP et HTTPS

HTTP est le protocole du Web. HTTPS est sa version chiffrée. Le cadenas dans la barre d'adresse garantit que **la communication est chiffrée (grâce au chiffrement TLS).**.



À retenir

Il ne garantit *pas* que le site est honnête : un site de phishing peut parfaitement être en HTTPS.

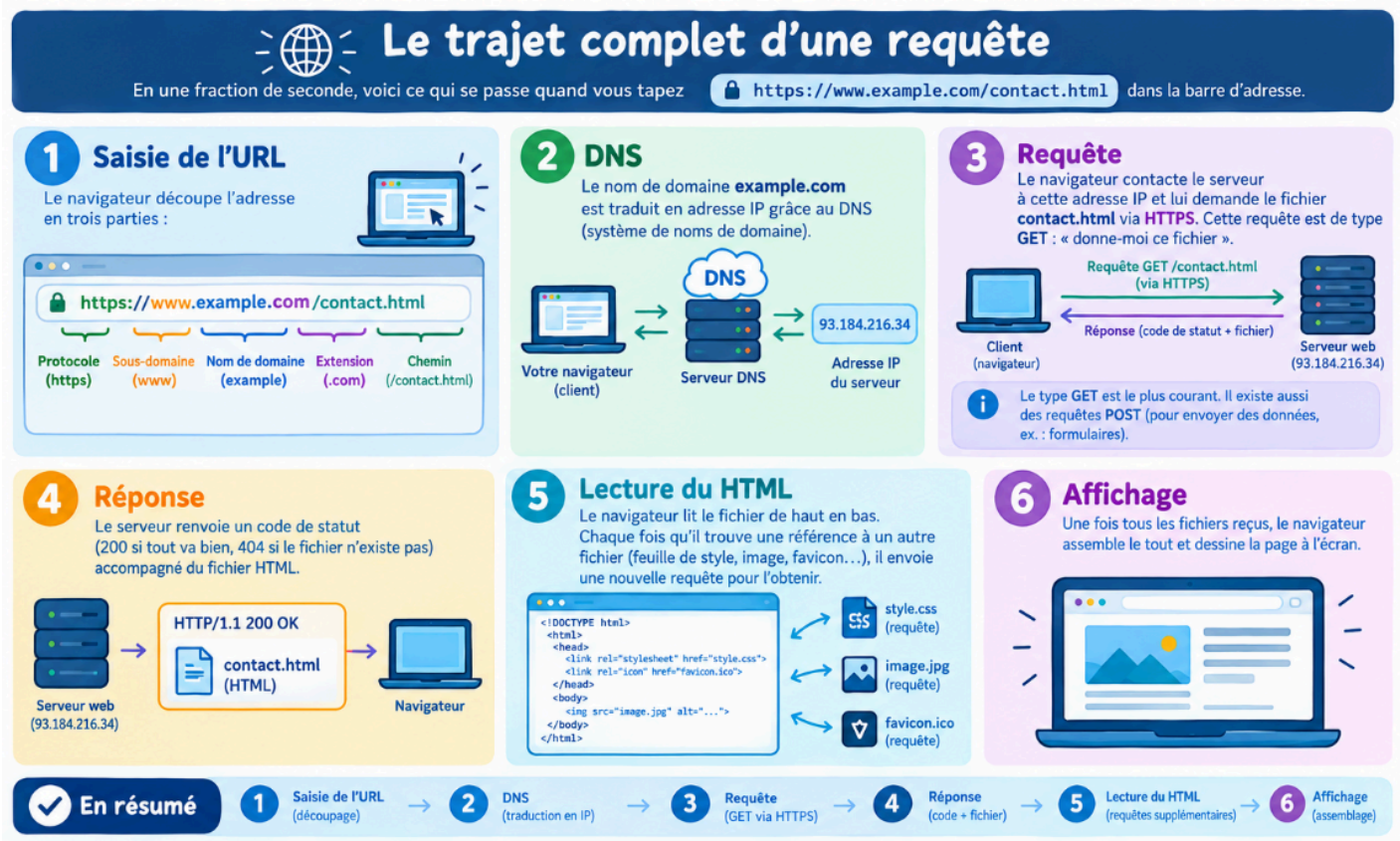
Quelques exemples de codes de réponse que vous croiserez — inutile de tous les mémoriser dès cette séance :

Codes de statut HTTP

Code	Signification
200	Tout va bien, voici la page
301	Redirection permanente : la page a déménagé définitivement
302	Redirection temporaire
403	Interdit : vous n'avez pas le droit d'accéder à cette page
404	Introuvable : la page n'existe pas
500	Erreur du serveur

Le trajet complet d'une requête

Assemblons les pièces. Voici ce qui se passe, en une fraction de seconde, quand vous tapez `https://www.example.com/contact.html` dans la barre d'adresse :



1. **Saisie de l'URL** : le navigateur découpe l'adresse (protocole, nom de domaine, chemin).
2. **DNS** : le nom `www.example.com` est traduit en adresse IP.
3. **Requête** : le navigateur (client) contacte le serveur à cette adresse et lui demande le fichier `contact.html` via HTTPS. Cette requête est de type **GET** : « donne-moi ce fichier ». C'est le type de requête le plus courant, celui qu'envoie le navigateur chaque fois que vous ouvrez une page ou cliquez sur un lien. Il en existe d'autres, comme **POST** pour envoyer des données : nous les verrons avec les formulaires. `contact.html` via HTTPS.
4. **Réponse** : le serveur renvoie un code de statut (200 si tout va bien, 404 si le fichier n'existe pas) accompagné du fichier HTML.
5. **Lecture du HTML** : le navigateur lit le fichier de haut en bas. Chaque fois qu'il y trouve une référence à un autre fichier — feuille de style, image, favicon —, il envoie **une nouvelle requête** pour l'obtenir.
6. **Affichage** : une fois les fichiers reçus, le navigateur assemble le tout et dessine la page à l'écran.

> ### Pourquoi l'étape 5 est importante pour vous
>
> Une page web n'est presque jamais un seul fichier : c'est un fichier HTML qui en appelle d'autres. Si le chemin vers une image est faux, seule cette requête échoue (code 404) : la page s'affiche quand même, avec une image cassée. C'est pour cela que les chemins, vus plus loin dans cette séance, sont si importants.
>
> **À essayer :** sur n'importe quel site, ouvrez les outils de développement (`F12`), onglet *Réseau*, puis rechargez la page. Vous verrez défiler chaque fichier demandé, avec son code de statut.

Le rôle du navigateur

Une page web n'a rien de magique : c'est **un simple fichier texte**, que n'importe qui peut lire. Le navigateur est le programme qui interprète ce texte et le transforme en page affichée. Deux réflexes à prendre dès aujourd'hui :

- `Ctrl+U` (afficher le code source) : montre le fichier HTML tel qu'il a été reçu du serveur.
- `F12` (outils de développement, onglet *Éléments* ou *Inspecteur*) : montre la page telle que le navigateur l'a construite, et permet de la modifier en direct. Rassurez-vous : ces modifications n'existent que sur votre écran, il suffit de recharger la page pour tout retrouver.

3. Le HTML

Hyper**T**ext **M**arkup **L**anguage : un langage de *balisage*. Il décrit la structure d'un document. Il ne contient ni variables, ni conditions, ni boucles : **ce n'est pas un langage de programmation**.

Trois langages, trois rôles

Le HTML n'est que l'un des trois langages du navigateur, et chacun a un rôle précis, et un seul. C'est un principe fondamental du Web : on **sépare** le contenu de sa présentation et de son comportement.

La séparation des rôles, avec l'image d'une maison

Langage	Rôle	Dans une maison
HTML	La structure et le sens du contenu : ceci est un titre, ceci est un paragraphe, ceci est un lien	Les murs et les pièces
CSS	La présentation : couleurs, tailles, polices, disposition	La peinture et la décoration
JavaScript	Le comportement : réagir aux clics, modifier la page	L'électricité et la domotique

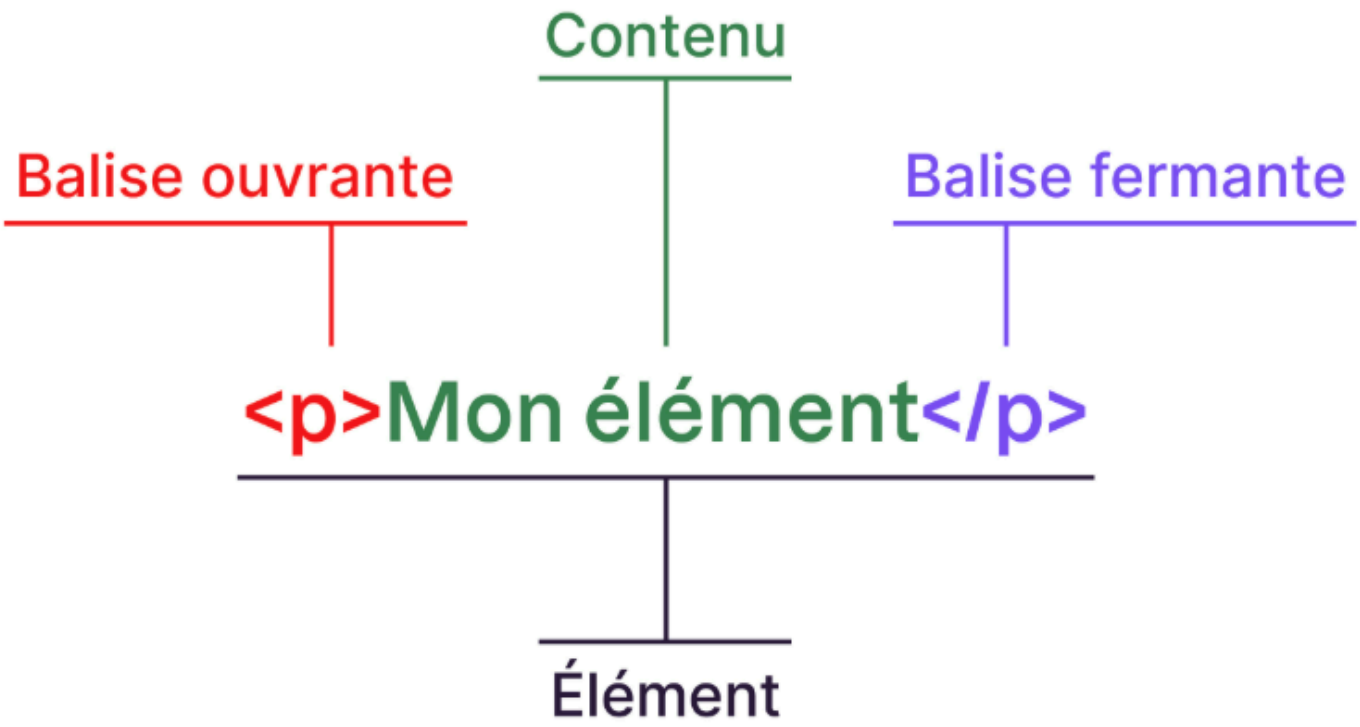
Conséquence directe : on ne choisit **jamais** une balise HTML pour son apparence. On la choisit pour ce que le contenu *est*. On peut repeindre une maison sans abattre ses murs ; de même, on doit pouvoir changer tout le CSS d'un site sans toucher à son HTML. Cette séance se concentre sur les murs : le HTML.

> **Attention**
>
> Dans ce cours, nous ne verrons que le **HTML** et le **CSS**. Le **JavaScript** est un langage plus complexe, qui sera abordé dans le cours de Projet de Développement Web en BA2.

Les balises

Il en existe deux sortes.

1. Les balises par paire : elles encadrent un contenu, avec une balise ouvrante et une balise fermante.



- **La balise ouvrante :** il s'agit du nom de l'élément (dans cet exemple, p pour paragraphe), encadré par des chevrons ouvrant et fermant. Cette balise ouvrante indique où l'élément commence ou commence à prendre effet. Dans cet exemple, elle précède le début du texte du paragraphe.
- **Le contenu :** il s'agit du contenu de l'élément. Dans cet exemple, c'est le texte du paragraphe : « Mon élément».

- **La balise fermante :** c'est la même que la balise ouvrante, sauf qu'elle comprend une barre oblique avant le nom de l'élément. Elle indique où l'élément se termine. Oublier d'inclure une balise fermante est une erreur fréquente chez les débutants qui peut produire des résultats étranges.

Exemple en HTML :

```
<!-- Balise par paire : elle entoure un contenu -->
<p>Ceci est un paragraphe.</p>
```

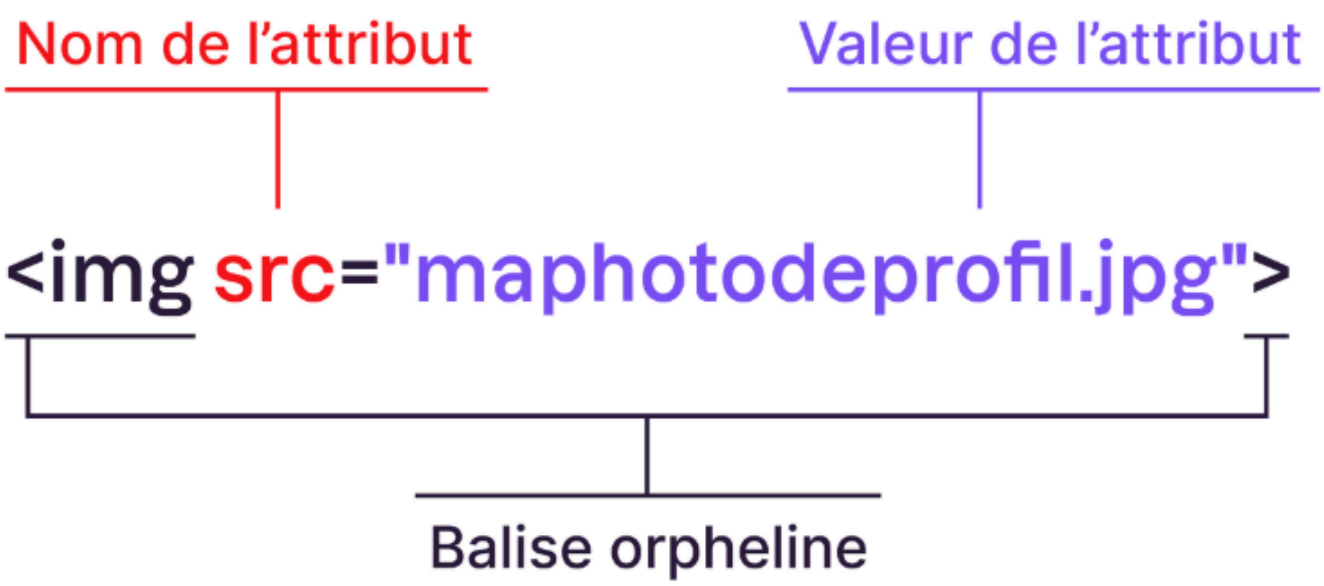
Exemple en rendu navigateur :

Ceci est un paragraphe.

2. Les balises orphelines : elles insèrent quelque chose, sans contenu. Elles n'ont pas de balise fermante.

Exemples de balises orphelines :

- `<br>` (saut de ligne)
- `<img>` (image)
- `<hr>` (ligne horizontale)



Exemple en HTML :

```
<!-- Balise orpheline : elle insère quelque chose, sans contenu -->
<br> <!-- Saut de ligne -->
<img src="photo.jpg" alt="Description de la photo"> <!-- Image -->
```

Exemple en rendu navigateur :

Un **attribut** ajoute une information à une balise, sous la forme `nom="valeur"`, à l'intérieur de la balise ouvrante. Il précise ou configure l'élément : quelle image afficher, quel texte proposer à la place si elle ne charge pas, vers quelle page pointer un lien, etc.

Un attribut doit avoir :

- Un espace entre lui et le nom de l'élément. Quand un élément possède plusieurs attributs, les attributs doivent aussi être séparés par des espaces.
- Le nom de l'attribut, suivi d'un signe égal (=).
- Une valeur d'attribut, entourée par des guillemets d'ouverture et de fermeture.

L'élément `<img>` peut recevoir plusieurs attributs, dont :

- `src` : un attribut obligatoire qui définit l'URL de l'image. Par exemple : src="https://mdn.github.io/shared-assets/images/examples/fx-nightly-512.png".
- `alt` : définit une description textuelle pour décrire l'image aux personnes qui ne peuvent pas la voir. Par exemple : `alt="Icône Firefox Nightly"`. Cet attribut n'est pas techniquement obligatoire, mais vous devez toujours fournir une description textuelle pour toutes les images qui véhiculent un sens (et non pour celles purement décoratives).
- `width` : définit la largeur de l'image en pixels. Par exemple : width="300".
- `height` : définit la hauteur de l'image en pixels. Par exemple : height="300".

Dans `<img src="photo.jpg" alt="Description de la photo">`, il y a deux attributs : `src` (la source — le fichier à afficher) et `alt` (le texte alternatif — lu par les lecteurs d'écran, et affiché si l'image ne se charge pas).

> ### Une note historique
>
> Vous rencontrerez parfois l'écriture `<br />` avec une barre oblique. C'est la syntaxe de XHTML, une variante des années 2000. En HTML5, on écrit simplement `<br>`. Les deux sont acceptées, mais autant prendre tout de suite la bonne habitude.

`<hr>` : une séparation thématique

Une autre balise orpheline : `<hr>` (*horizontal rule*) marque une rupture thématique dans le contenu, par exemple entre deux sujets différents d'une même page. Le navigateur l'affiche par défaut comme un trait horizontal, mais son rôle est **sémantique**, pas décoratif : ce n'est pas un outil pour « tracer une ligne », c'est le travail du CSS.

Exemple en HTML :

```
<p>Fin du premier chapitre.</p>
<hr>
<p>Début du second chapitre.</p>
```

Exemple en rendu navigateur :

L'imbrication

Les balises s'emboîtent comme des poupées russes. On ferme toujours dans l'ordre inverse de l'ouverture.

L'exemple ci-dessous utilise `<strong>`, qui marque une importance forte (il s'affiche en gras par défaut). Vous le détaillerez, avec son équivalent `<em>`, à la séance 2.

Exemple en HTML :

```
<!-- Correct -->
<p>Un texte <strong>important</strong> ici.</p>

<!-- Incorrect : les balises se croisent -->
<p>Un texte <strong>important</p></strong>
```

Exemple en rendu navigateur :

Un texte **important** ici.

Un texte **important**

Les entités HTML

Certains caractères ont un sens particulier en HTML (`<` , `>` , `&` ...) : vous ne pouvez pas les taper tels quels dans votre texte, sinon le navigateur croit que vous ouvrez une balise. Pour les afficher, on utilise une **entité** : un code qui commence par `&` et se termine par `;` .

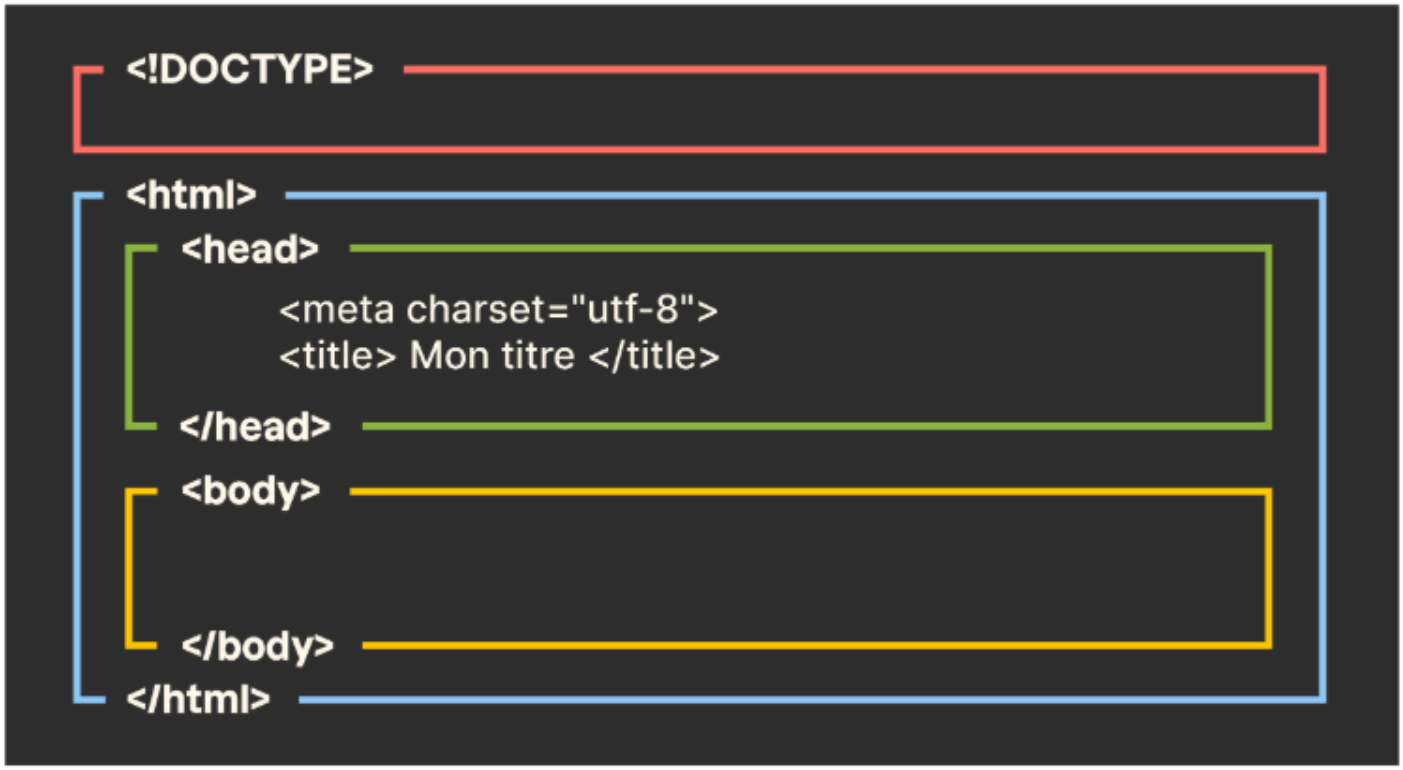
Entités HTML courantes

On écrit	Le navigateur affiche
`&`	& (esperluette)
`<`	< (signe inférieur)
`>`	> (signe supérieur)
`©`	© (copyright)
` `	un espace insécable (invisible)
`"`	" (guillemet)
`'`	' (apostrophe)

Le squelette d'une page

Le contenu visible (vos titres, vos paragraphes) ne suffit pas : le navigateur a besoin d'informations *sur* la page avant même de l'afficher — quel encodage utiliser pour lire les caractères, quel titre montrer dans l'onglet, à quelle feuille de style se connecter. C'est le rôle du squelette : un emballage fixe, toujours le même, dans lequel vient se loger votre contenu.

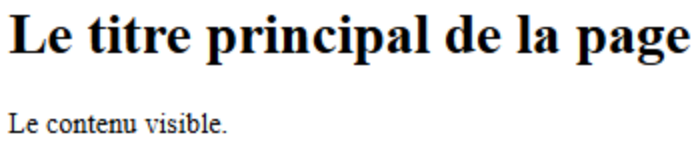
Toute page HTML suit ce squelette. Apprenez-le par cœur.



Exemple en HTML :

```
<!DOCTYPE html>
<html lang="fr">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="Une phrase décrivant la page.">
  <title>Le titre affiché dans l'onglet</title>
  <link rel="icon" href="images/favicon.svg">
  <link rel="stylesheet" href="css/base.css">
</head>

<body>

  <h1>Le titre principal de la page</h1>
  <p>Le contenu visible.</p>

</body>

</html>
```

Exemple en rendu navigateur :



4. Lire et comprendre une page HTML

Reprenons le squelette ligne par ligne :

Ce que fait chaque ligne

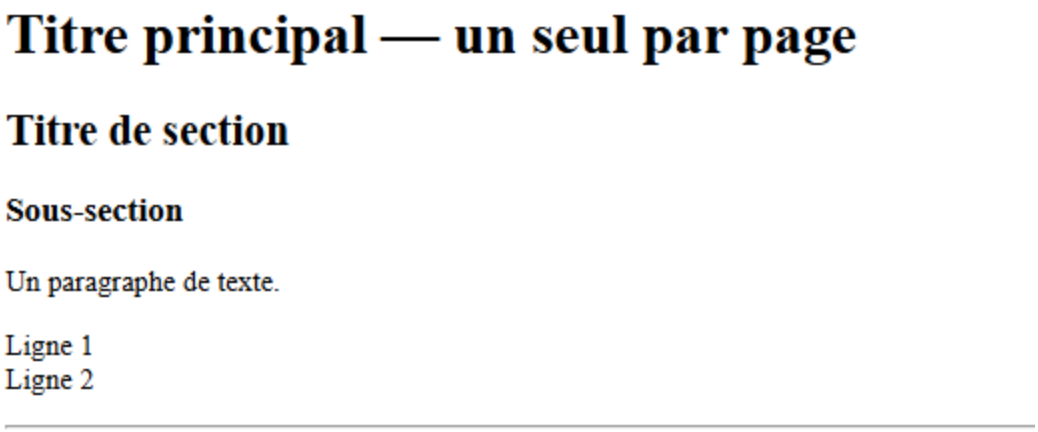
Élément	Rôle
`<!DOCTYPE html>`	Indique au navigateur qu'il s'agit de HTML5. Sans elle, le navigateur bascule dans un mode de compatibilité ancien et l'affichage devient imprévisible.
`<html>`	L'élément <html>. Cet élément englobe tout le contenu de la page. Il est parfois appelé l'élément racine.
`<html lang="fr">`	L'élément racine : il contient toute la page (le `<head>` et le `<body>`). L'attribut `lang` donne la langue du document, utilisée par les lecteurs d'écran pour choisir la bonne prononciation, et par les moteurs de recherche.
`<head>`	Cet élément sert de conteneur pour les informations sur la page **ne font pas partie du contenu visible par les utilisateurs**. Cela peut inclure des mots-clés et une description de la page à afficher dans les résultats de recherche, du CSS pour la mise en forme du contenu, des déclarations d'encodage de caractères, et plus encore. Vous en apprendrez davantage sur l'en-tête de la page dans une séance à venir.
`charset="UTF-8"`	Encodage des caractères. Sans cette ligne, les accents s'affichent en charabia.
`viewport`	Indispensable pour l'affichage sur téléphone.
`<meta name="description">`	Un résumé de la page en une phrase, affiché sous le titre dans les résultats Google.
`<title>`	Il définit le titre de la page, c'est-à-dire le titre qui apparaît dans l'onglet du navigateur où la page est chargée. Le titre de la page est également utilisé pour décrire la page lorsqu'elle est ajoutée aux favoris.
`<link rel="icon">`	La petite icône affichée dans l'onglet du navigateur (le favicon).
`<link rel="stylesheet">`	Relie la page à sa feuille de style CSS, qui définit son apparence — vous verrez ça en détail à partir de la séance 5.
`<body>`	Tout ce qui est visible. Il contient tout le contenu affiché sur la page, y compris le texte, les images, les vidéos, les jeux, les pistes audio lisibles, etc.

Les titres et les paragraphes

Exemple en HTML :

```
<h1>Titre principal — un seul par page</h1>
<h2>Titre de section</h2>
<h3>Sous-section</h3>

<p>Un paragraphe de texte.</p>
<p>Ligne 1<br>Ligne 2</p>

<hr>
```

Exemple en rendu navigateur :



Il vous revient de décider ce que représentent les éléments utilisés, tant que la hiérarchie a du sens. Gardez simplement à l'esprit quelques bonnes pratiques lors de la création de telles structures :

De préférence, vous devez utiliser un seul `<h1>` par page — c'est le titre de niveau supérieur, tous les autres se situent en dessous dans la hiérarchie. Veillez à utiliser les titres dans le bon ordre hiérarchique. N'utilisez pas des éléments `<h3>` pour représenter des sous-titres, suivis d'éléments `<h2>` pour représenter des sous-sous-titres — cela n'a pas de sens et produit des résultats étranges. Parmi les six niveaux de titres disponibles, essayez d'en utiliser au maximum trois par page, sauf si cela s'avère nécessaire. Les documents avec de nombreux niveaux (par exemple, une hiérarchie de titres profonde) deviennent difficiles à manipuler et à parcourir. Dans ce cas, il est conseillé de répartir le contenu sur plusieurs pages si possible.

> ### Les titres ne sont pas une question de taille
>
> On ne choisit pas `<h2>` parce qu'on veut du texte un peu plus petit. On le choisit parce que c'est une **section de niveau 2**. La taille, c'est le travail du CSS.
>
> Deux règles : un seul `<h1>` par page, et jamais de saut de niveau — pas de `<h3>` directement après un `<h1>`.

Les commentaires

Exemple en HTML :

```
<!-- Ceci est un commentaire : le navigateur l'ignore -->
```

Ils servent à trois choses :

- Expliquer une partie du code
- Désactiver temporairement du code sans le supprimer
- Se repérer dans un fichier long

Commentez ce qui est complexe. Ne commentez pas l'évidence. Et souvenez-vous qu'un commentaire HTML est **visible** par quiconque affiche le code source : n'y écrivez jamais rien de confidentiel.

L'organisation des fichiers

```
mon-site/
├── index.html      ← le fichier d'accueil s'appelle TOUJOURS index.html
├── css/            ← les feuilles de style
├── images/         ← les images
└── medias/         ← les sons et vidéos
```

Les liens hypertextes et les ressources

Une page HTML ne vit jamais seule : elle référence d'autres fichiers, et chacune de ces références déclenche sa propre requête vers le serveur (vu plus haut, à l'étape 5 du trajet d'une requête). Trois balises à connaître :

- `<a href="...">` : un lien vers une autre page, sur votre site ou ailleurs.

- `<img src="..." alt="...">` : une image à afficher.
- `<link rel="stylesheet" href="...">` : une feuille de style à appliquer, placée dans le `<head>`.

Dans les trois cas, l'attribut qui pointe vers le fichier (`href` ou `src`) suit exactement les mêmes règles de chemin, détaillées ci-dessous.

Chemins absolus et chemins relatifs

C'est le point le plus important de la séance, et la première cause d'erreur de tout le cours. Imaginons cette arborescence :

```
mon-site/
├── index.html
├── css/
│   └── base.css
├── images/
│   └── photo.jpg
└── pages/
    └── contact.html
```

Depuis `index.html` :

Écriture des chemins depuis index.html

Pour atteindre	On écrit	Lecture
La feuille de style	`css/base.css`	dans le dossier css, à côté de moi
La photo	`images/photo.jpg`	dans le dossier images, à côté de moi
La page contact	`pages/contact.html`	dans le dossier pages, à côté de moi

Mais depuis `pages/contact.html`, pour revenir à l'accueil :

```
<a href="../index.html">Accueil</a>
```

Les deux points `..` signifient **remonter d'un dossier**. C'est la notation à retenir.

Tous les chemins ci-dessus sont **relatifs** : ils partent de l'endroit où se trouve le fichier en cours. Il existe deux autres façons d'écrire une adresse :

Les trois types de chemins

Type	Exemple	Quand l'utiliser
Relatif	`images/photo.jpg` ou `../images/photo.jpg`	Pour tous les fichiers de **votre** site — la forme exacte dépend de l'endroit où se trouve le fichier qui écrit le chemin (voir le tableau ci-dessus). C'est la règle dans ce cours.
Absolu (URL complète)	`https://developer.mozilla.org/fr/`	Pour un lien vers **un autre site**.
Depuis la racine	`/images/photo.jpg`	À éviter dans ce cours (voir ci-dessous).

Pourquoi éviter la barre oblique au début

Un chemin qui commence par `/` part de la racine du serveur, pas de celle de votre projet. Avec Live Server, tout semble fonctionner. Mais une fois votre site transféré par FTP, si votre hébergement le place dans un sous-dossier plutôt qu'à la racine, la racine du serveur n'est plus celle de votre site, et toutes vos images et feuilles de style disparaissent d'un coup.

Le piège qui vous coûtera une soirée

Sur votre ordinateur Windows, `Photo.JPG` et `photo.jpg` sont le même fichier. Sur le serveur qui hébergera votre site — une machine Linux —, ce sont **deux fichiers différents**.

D'où la règle : noms de fichiers **en minuscules, sans espaces, sans accents**. Un trait d'union à la place de l'espace.

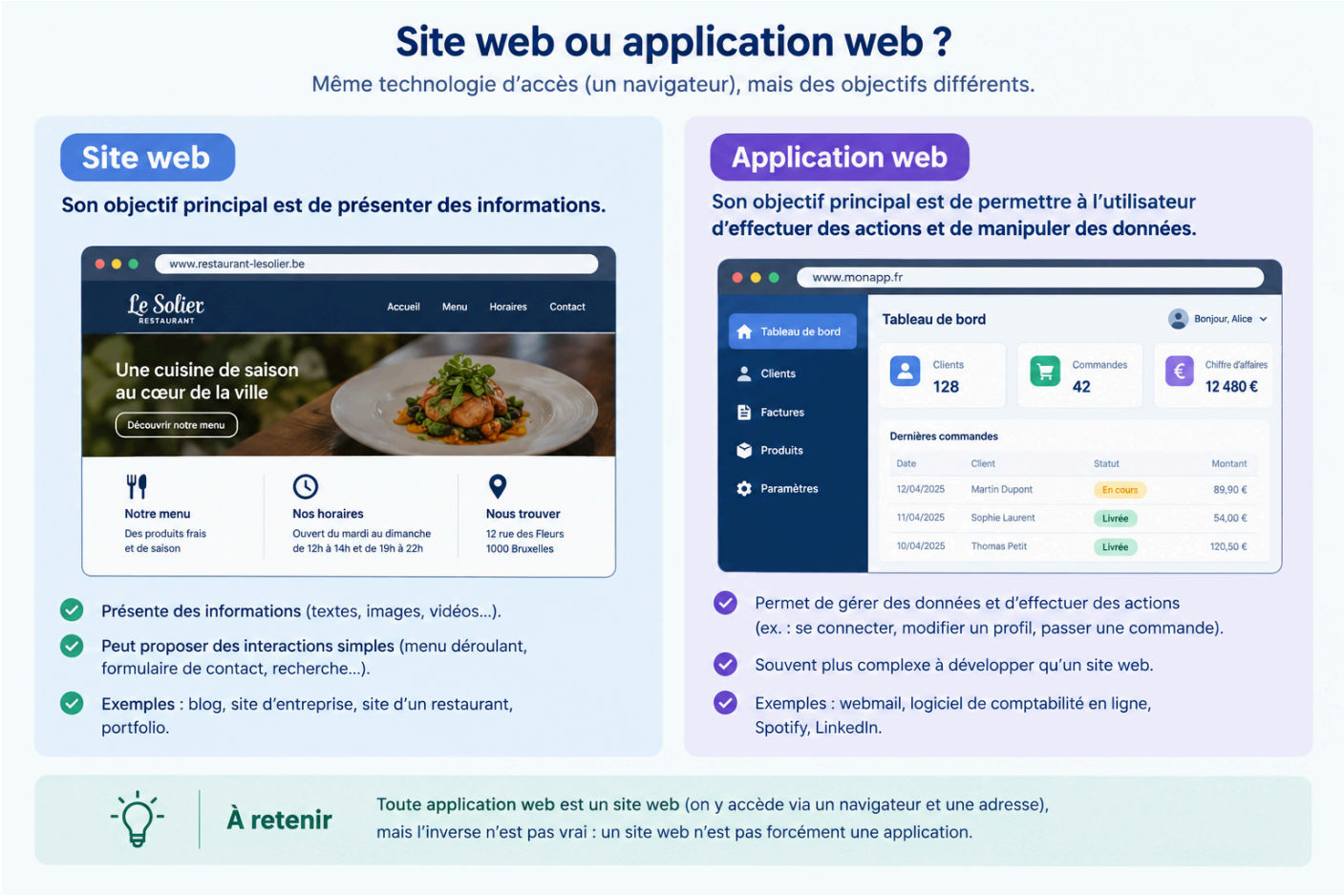
Accessibilité : les premiers réflexes

Vous recroiserez l'accessibilité en détail à la séance 8, mais trois réflexes s'installent dès maintenant, balise par balise :

- **Une hiérarchie de titres logique** : un seul `<h1>` par page, puis des `<h2>`, `<h3>`… sans sauter de niveau (vu ci-dessus).
- **Un `alt` pertinent** sur chaque image qui porte une information, pas sur une image purement décorative.
- **La langue du document déclarée** avec `lang="fr"` sur la balise `<html>` (vue dans le squelette de page).

5. Les types de sites web : usages et fonctionnement

Objectif et usage : Site web ou application web ?



Un **site web** a une vocation avant tout informative : son objectif principal est de présenter des informations aux visiteurs. C'est le cas du site d'un restaurant, qui affiche son menu, ses horaires et son adresse — même s'il peut aussi proposer quelques interactions simples, comme un formulaire de contact.

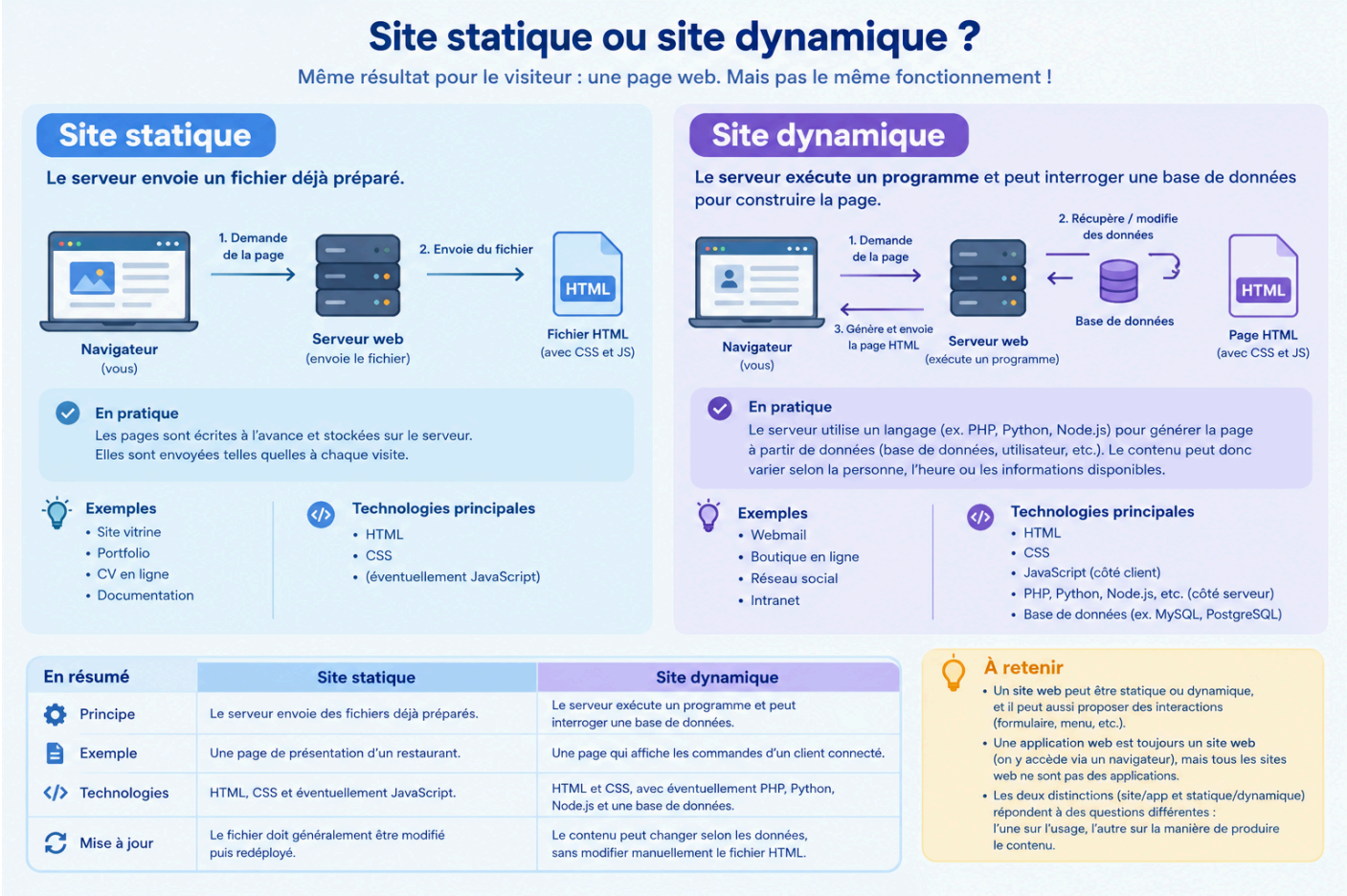
Une **application web** a une vocation différente : son objectif principal est de permettre à l'utilisateur d'effectuer des actions et de manipuler des données, le plus souvent après s'être connecté à un compte personnel. C'est le cas d'une application de comptabilité en ligne, qui permet d'enregistrer des factures, de calculer des totaux et de consulter des

rapports — ou de services comme un webmail, Spotify ou LinkedIn. Une application web est généralement plus complexe et plus coûteuse à développer qu'un site web.

Techniquement, **toute application web est un site web** (on y accède via un navigateur et une adresse), mais l'inverse n'est pas vrai. La frontière n'est cependant pas toujours nette : un site peut très bien intégrer des fonctionnalités d'application (une recherche, un compte utilisateur basique) sans devenir pour autant une application complète.

Attention cependant à ne pas mélanger les deux distinctions. « Site » et « application » parlent de l'**usage**, du point de vue du visiteur : consulter de l'information, ou manipuler des données. « Statique » et « dynamique », vus ci-dessous, parlent de la manière dont le contenu est **produit et servi** par le serveur — une question technique, pas d'usage. Un site statique peut très bien proposer des interactions dans le navigateur (un menu qui se déplie, un formulaire qui se valide) grâce au JavaScript côté client, sans pour autant devenir une application ni dépendre d'un serveur dynamique.

Fonctionnement technique : Site statique ou site dynamique ?



Les deux grandes familles de sites

	Site statique	Site dynamique
Principe	Le serveur envoie des fichiers déjà préparés.	Le serveur peut exécuter un programme et récupérer des données pour construire la réponse.
Exemple	Une page de présentation d'un restaurant.	Une page qui affiche les commandes d'un client connecté.
Technologies	HTML, CSS et éventuellement JavaScript.	HTML et CSS, avec éventuellement PHP, Python, Node.js et une base de données.
Mise à jour	Le fichier doit généralement être modifié puis redéployé.	Le contenu peut changer selon les données, sans modifier manuellement le fichier HTML pour chaque changement.

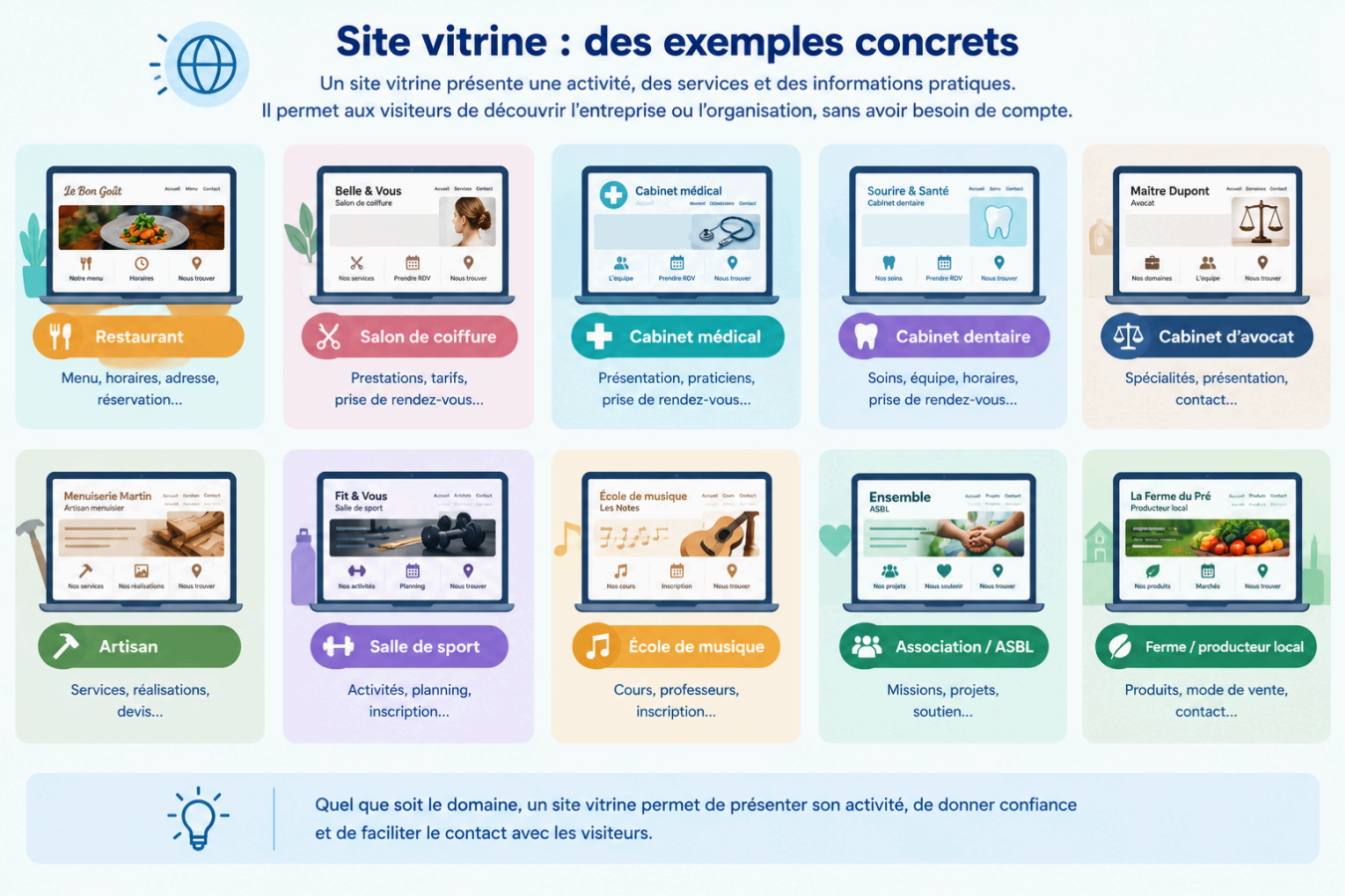
> ### Le point important
>
> Un site dynamique produit **quand même** du HTML et du CSS à l'arrivée dans le navigateur. Le dynamique s'ajoute par-dessous, il ne remplace rien. C'est pourquoi le statique est le socle, et c'est pourquoi il est enseigné en premier.

Quand choisir l'un ou l'autre

Le choix dépend du volume de contenu, de la fréquence de mise à jour, du besoin d'interaction avec l'utilisateur, et du budget d'hébergement disponible. Un fil conducteur simple : **si le contenu ne change pas souvent et ne dépend pas d'un compte utilisateur ou d'une base de données, un site statique suffit**.

Quelques cas d'usage typiques d'un site statique :

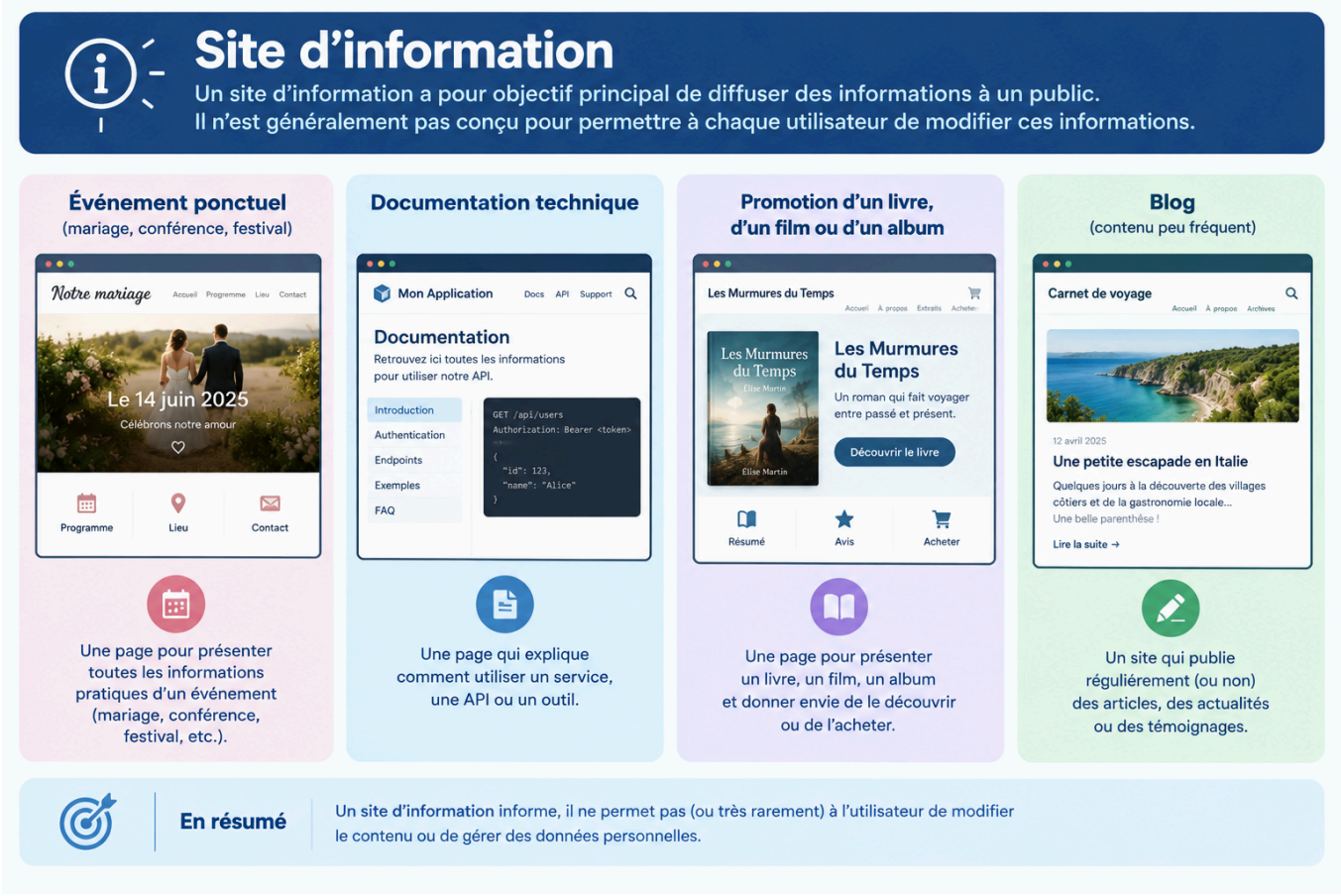
- **Site vitrine** : restaurant, salon de coiffure, cabinet médical, dentaire ou d'avocat, artisan (plombier, électricien, menuisier), salle de sport, école de musique, association ou ASBL, ferme ou producteur local.



Prendre rendez-vous :

- **Statique : Formulaire de contact** : simple formulaire pour envoyer un message au propriétaire du site.
- **Dynamique : Réservation dans un agenda** : le visiteur choisit une date et une heure parmi les créneaux disponibles. Le système vérifie les disponibilités et enregistre le rendez-vous sur le serveur, afin d'éviter les doubles réservations.

- **Site d'information** : page d'un événement ponctuel (mariage, conférence, festival), documentation technique, page de promotion d'un livre, d'un film ou d'un album, blog à contenu peu fréquent.



- **Landing page** : présentation d'un produit ou d'une application avant son lancement, page « coming soon », page de mémoire ou de TFE.

Landing page : des exemples concrets

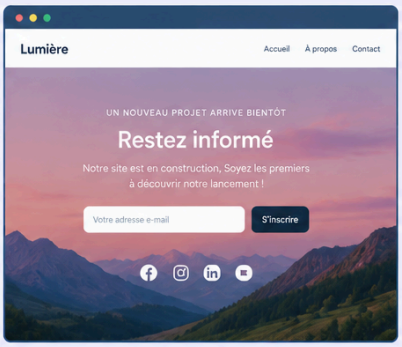
Une landing page est une page web simple et ciblée, conçue pour présenter un produit, une application ou un projet, et inciter le visiteur à passer à l'action.

Présentation d'un produit ou d'une application

Objectif

Présenter un produit ou une application avant sa sortie, et susciter l'intérêt des visiteurs (en s'inscrivant, en précommandant, etc.).

Page « coming soon »



Objectif

Prévenir les visiteurs du lancement prochain d'un site, d'un service ou d'un événement, et récupérer leurs coordonnées.

Page de mémoire ou de TFE

Objectif

Présenter un travail académique (mémoire, TFE), expliquer la problématique, la méthode et les résultats, et permettre d'y accéder facilement.

À retenir

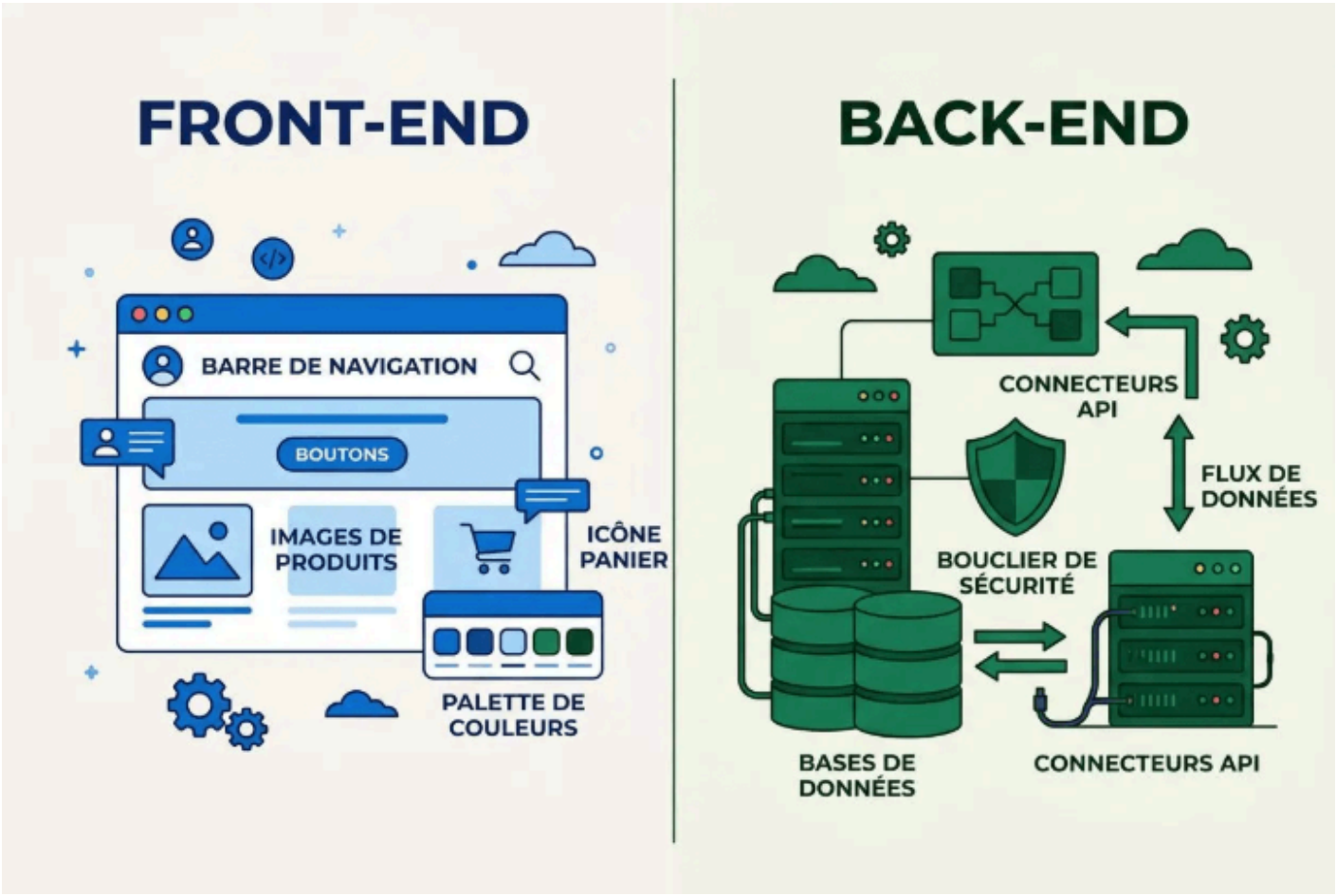
Une landing page est souvent plus simple qu'un site complet : elle se concentre sur un seul objectif et guide le visiteur vers une action précise.

- **Portfolio ou CV** : photographe, designer, illustrateur, CV en ligne.

Les projets de ce cours

Les trois projets que vous réaliserez cette année sont des sites **statiques**, en HTML et CSS purs : c'est le socle indispensable avant d'aborder le dynamique dans d'autres UE.

Côté client et côté serveur



- **Front-end** (dans votre navigateur) : HTML pour la structure, CSS pour la présentation, JavaScript pour l'interactivité
- **Back-end** (sur le serveur) : PHP, Python, Node.js, Java, et un système de base de données

Cette UE couvre **HTML et CSS**. JavaScript sera abordé dans *Projet de développement web*, les bases de données dans *Initiation aux bases de données*.

6. Normes et validation

Les normes du Web

Sans règles communes, chaque navigateur interpréterait le code à sa façon, et un même site s'afficherait différemment partout. Ces règles sont écrites par deux organismes :

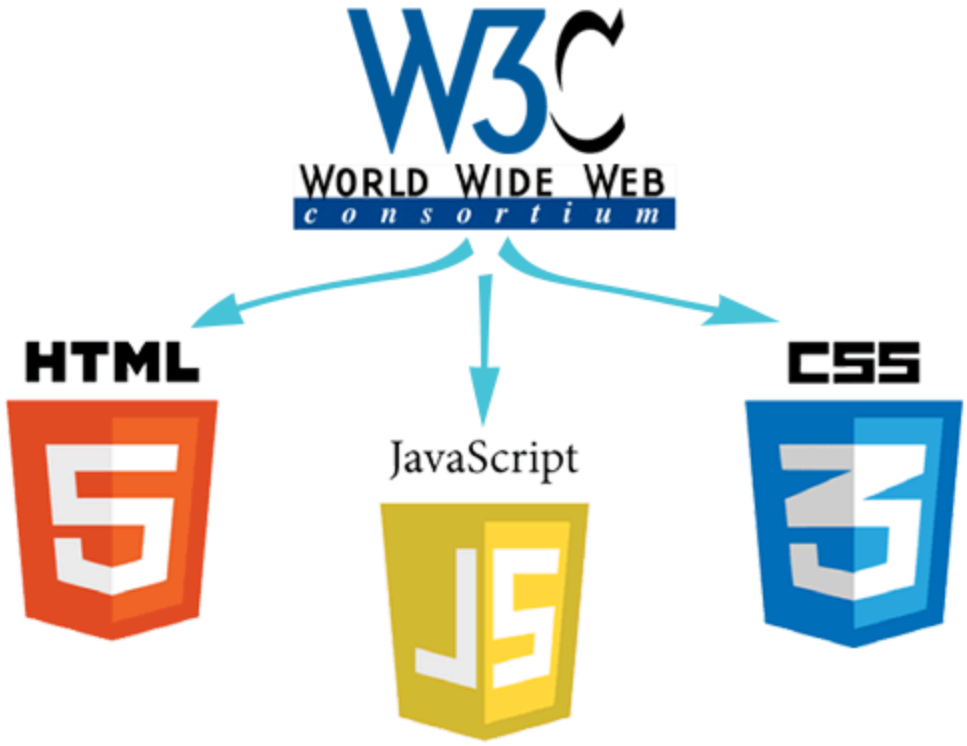
- Le W3C, fondé en 1994 par Tim Berners-Lee, qui écrit notamment les standards du CSS et de l'accessibilité (les recommandations qui aident à rendre les sites utilisables par les personnes en situation de handicap), et qui fournit le validateur que vous utiliserez dans ce cours.
- Le WHATWG, qui réunit les éditeurs de navigateurs (Apple, Google, Mozilla, Microsoft) et maintient aujourd'hui le standard du HTML. Depuis 2019, il n'existe plus de « HTML5 » figé : le HTML est un **standard vivant** (*HTML Living Standard*), mis à jour en continu.

Respecter les standards augmente les chances qu'une page reste **compatible** avec les différents navigateurs, qu'elle **continue de fonctionner** dans plusieurs années, et qu'elle

soit plus facilement **accessible** aux personnes qui utilisent un lecteur d'écran ou un autre outil d'assistance.

Valider son code

Le navigateur est très tolérant : il affiche votre page même si elle contient des erreurs. C'est un piège! vous croyez que tout va bien.



L'outil de référence est le validateur du W3C. Prenez l'habitude d'y passer chaque page : il détecte les erreurs de conformité HTML (balises mal fermées, attributs manquants, structure invalide…). Un code valide **n'est pas pour autant une garantie absolue**, il reste possible qu'une page valide s'affiche mal dans un navigateur particulier, ou pose des problèmes d'accessibilité que le validateur ne détecte pas. C'est un point de départ fiable, pas un contrôle final.

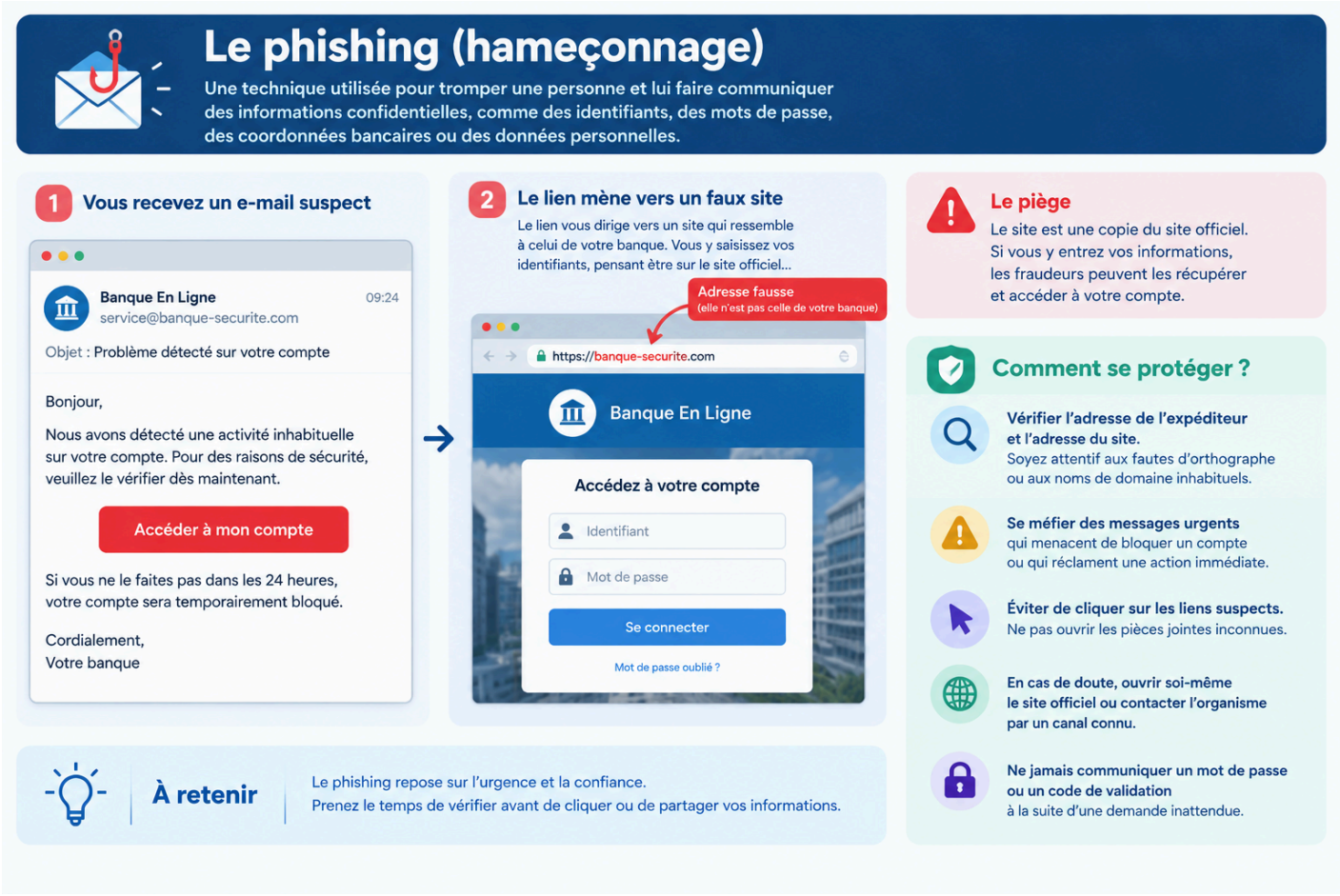
7. La sécurité de base

Le Web nous permet de communiquer, de nous informer et d'effectuer de nombreuses démarches. Cependant, tous les sites et tous les messages ne sont pas fiables. Certains cherchent à voler des informations, à installer des programmes malveillants ou à utiliser du contenu sans autorisation.

Il est donc important de connaître quelques risques courants et de savoir comment s'en protéger.

1. Le phishing (hameçonnage)

Le **phishing**, ou hameçonnage, est une technique utilisée pour tromper une personne afin qu'elle communique des informations confidentielles : identifiants, mots de passe, coordonnées bancaires ou données personnelles.



Exemple : vous recevez un e-mail qui semble provenir de votre banque. Le message indique qu'un problème a été détecté sur votre compte et vous demande de cliquer sur un lien pour le sécuriser. Le lien mène en réalité vers un faux site qui ressemble à celui de votre banque. Si vous y saisissez votre mot de passe, celui-ci peut être récupéré par les fraudeurs.

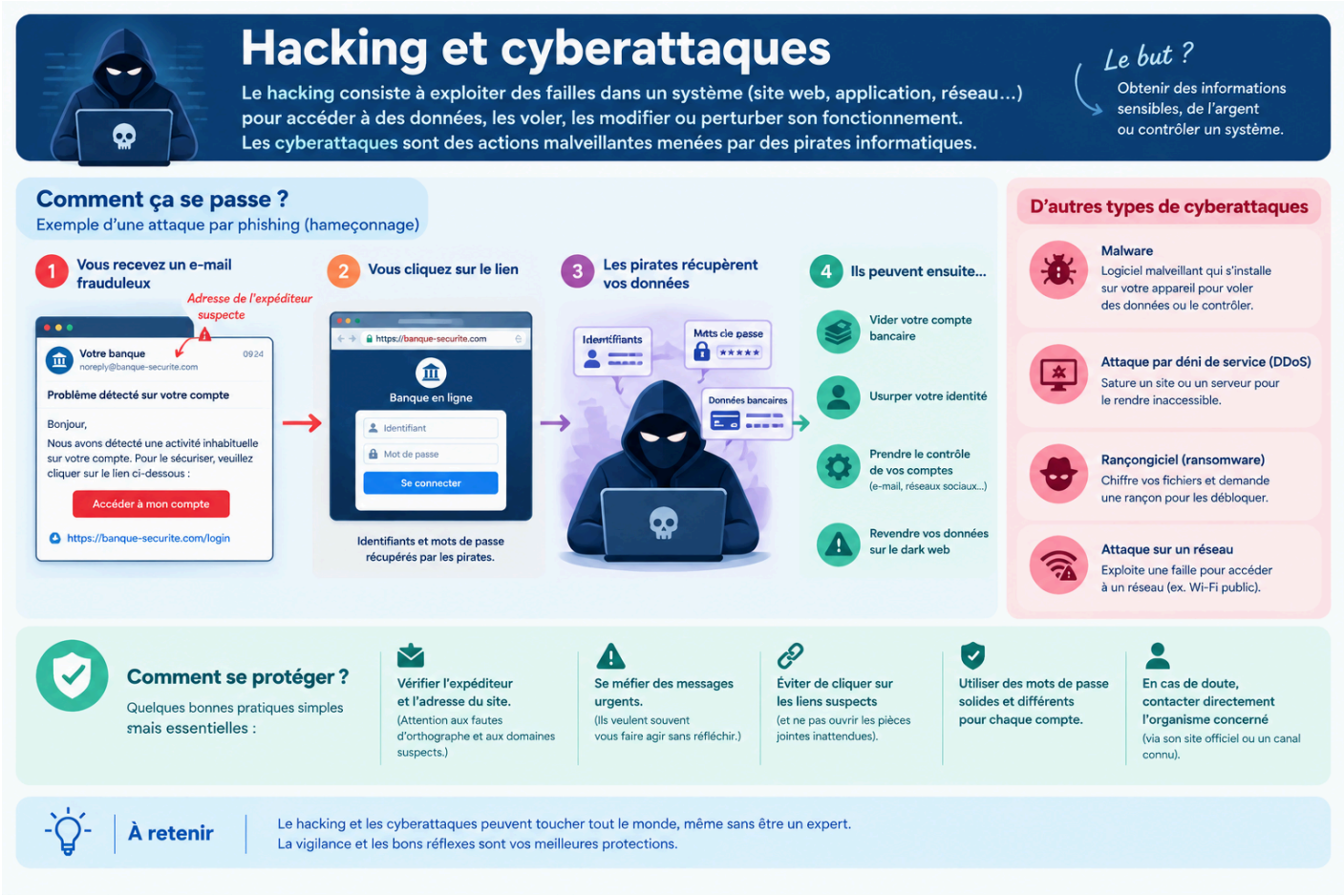
Comment se protéger ?

- Vérifier attentivement l'adresse de l'expéditeur et l'adresse du site.
- Se méfier des messages urgents qui menacent de bloquer un compte ou réclament une action immédiate.
- Éviter de cliquer sur les liens suspects.
- En cas de doute, ouvrir soi-même le site officiel ou contacter l'organisme par un canal connu.

- Ne jamais communiquer un mot de passe ou un code de validation à la suite d'une demande inattendue.

2. Le hacking et les cyberattaques

Le terme **hacking** désigne différentes pratiques liées à l'accès, à l'analyse ou à la modification de systèmes informatiques. Dans le contexte de la sécurité, il peut désigner l'exploitation d'une vulnérabilité pour accéder à un système sans autorisation.



Une **vulnérabilité** est une faiblesse dans un logiciel, un site Web, une configuration ou un système informatique. Lorsqu'elle n'est pas corrigée, une personne malveillante peut parfois l'exploiter pour consulter des données, modifier du contenu ou perturber le fonctionnement du système.

Exemple : un site Web utilise une ancienne version d'un logiciel qui contient une faille connue. Un attaquant exploite cette faille pour accéder à des informations auxquelles il ne devrait pas avoir accès.

Comment se protéger ?

- Installer les mises à jour des logiciels et des systèmes.
- Utiliser des mots de passe robustes et différents selon les services.

- Activer l'authentification à deux facteurs lorsque c'est possible.
- Limiter les accès aux personnes qui en ont réellement besoin.
- Pour les développeurs, corriger les vulnérabilités et protéger les données traitées par le site.

> **À retenir**
>
> le hacking n'est pas toujours malveillant. Des spécialistes réalisent aussi des tests de sécurité avec une autorisation explicite afin de détecter les failles avant qu'elles ne soient exploitées.

3. Les virus et les autres logiciels malveillants

Un **virus informatique** est un type de logiciel malveillant capable de se reproduire en s'insérant dans d'autres fichiers ou programmes. Il peut se propager lorsque ces fichiers sont utilisés ou partagés.

Il existe également d'autres logiciels malveillants, regroupés sous le terme *malware* :

- **Cheval de Troie (trojan) :** un programme qui semble légitime, mais qui cache une fonction malveillante.
- **Rançongiciel (ransomware) :** un logiciel qui bloque l'accès à des fichiers ou à un système et réclame une rançon.
- **Logiciel espion (spyware) :** un programme qui collecte des informations sur l'utilisateur à son insu.

Ces logiciels peuvent endommager un système, voler des informations, surveiller une activité ou rendre des fichiers inaccessibles.

Exemple : une personne télécharge un logiciel depuis un site non fiable. Le fichier contient aussi un programme malveillant qui tente de récupérer des informations présentes sur son ordinateur.

Comment se protéger ?

- Télécharger les logiciels depuis des sources officielles ou fiables.
- Ne pas ouvrir les pièces jointes inattendues.
- Maintenir le système et les logiciels à jour.
- Utiliser les protections de sécurité disponibles sur son appareil.
- Effectuer des sauvegardes régulières des fichiers importants.

4. Le grabbing et la copie de contenu

Le terme **grabbing** est parfois utilisé pour désigner la récupération ou la copie de contenu depuis un site Web : textes, images, fichiers ou autres éléments. Cette copie peut être automatisée à l'aide de programmes.

Il faut distinguer la récupération technique d'un contenu de son utilisation. Un contenu accessible sur Internet n'est pas nécessairement libre de droits. Copier et republier des textes, des images ou une mise en page sans autorisation peut porter atteinte aux droits d'auteur ou à d'autres droits applicables.

Exemple : une personne copie les photographies et les textes d'un site commercial pour les publier sur son propre site en faisant croire qu'il s'agit de ses créations.

Comment agir correctement ?

- Vérifier les droits associés au contenu avant de le réutiliser.
- Demander une autorisation lorsque cela est nécessaire.
- Respecter les conditions des licences, notamment pour les images et les textes.
- Créer ses propres contenus ou utiliser des ressources dont la licence autorise l'utilisation envisagée.
- Mentionner la source lorsque cela est requis, en sachant qu'une simple citation de la source ne remplace pas toujours une autorisation.

> **Attention**
>
> La copie de contenu n'est pas automatiquement illégale dans tous les cas. Cela dépend notamment du contenu, des droits applicables, de la licence et de l'utilisation qui en est faite.

5. Les bons réflexes à adopter sur le Web

Quelques habitudes simples permettent de réduire les risques :

- **Vérifier les adresses Web** avant de saisir des informations personnelles.
- **Utiliser des mots de passe robustes et uniques**, idéalement gérés avec un gestionnaire de mots de passe.
- **Activer l'authentification à deux facteurs** pour les comptes importants.
- **Installer les mises à jour** du navigateur, du système et des logiciels.
- **Faire attention aux téléchargements** et aux pièces jointes.
- **Limiter les informations personnelles publiées** en ligne.
- **Effectuer des sauvegardes** des fichiers importants.

Enfin, la présence de `https://` dans une adresse indique que la connexion entre le navigateur et le site est chiffrée. C'est important, pour rappel cela ne garantit pas que le site est honnête : un site frauduleux peut lui aussi utiliser HTTPS.

> **À retenir**
>
> La sécurité sur le Web repose à la fois sur des outils techniques, des logiciels à jour et le comportement prudent des utilisateurs.

Testez vos connaissances

Un petit tour de la séance, question par question. Essayez de répondre avant de cliquer pour voir la réponse.

Comprendre ce qu'est le Web

▼ **Quelle est la différence entre Internet et le Web ?**

Internet est l'infrastructure : le réseau mondial qui relie les machines entre elles. Le Web est l'un des services qui circulent sur ce réseau — au même titre que l'e-mail ou la visioconférence — basé sur des pages reliées par des liens hypertextes.

▼ **Quelle est la différence entre un site web et une page web ?**

Une page web est un document unique, avec sa propre URL. Un site web est un ensemble de pages reliées entre elles, regroupées sous un même nom de domaine.

Comment une page arrive à l'écran

▼ **À quoi sert le DNS ?**

À traduire un nom de domaine (`www.exemple.be`) en adresse IP, comme un annuaire téléphonique : on cherche un nom, on obtient un numéro.

▼ **Le cadenas HTTPS garantit-il qu'un site est honnête ?**

Non. Il garantit seulement que la communication est chiffrée. Un site de phishing peut parfaitement être en HTTPS.

Le HTML

▼ **Le HTML est-il un langage de programmation ?**

Non. Il ne contient ni variables, ni conditions, ni boucles : c'est un langage de balisage qui décrit la structure d'un document.

▼ **Quelle est la différence entre une balise par paire et une balise orpheline ?**

Une balise par paire entoure un contenu (`<p>...</p>`). Une balise orpheline insère quelque chose sans contenu (`<br>` , `<img>`).

Lire et comprendre une page HTML

▼ Pourquoi faut-il éviter les majuscules, espaces et accents dans les noms de fichiers ?

Parce que Windows ne fait pas la différence entre majuscules et minuscules, mais le serveur (une machine Linux), si : `Photo.JPG` et `photo.jpg` y sont deux fichiers différents.

▼ Que signifient les deux points `..` dans un chemin ?

Remonter d'un dossier.

Les types de sites web

▼ Un site statique peut-il proposer des interactions (menu déroulant, formulaire) ?

Oui, grâce au JavaScript côté navigateur. Cela ne le rend pas dynamique pour autant : le serveur envoie toujours le même fichier, écrit à l'avance.

▼ Quelle est la différence entre un site web et une application web ?

Un site web a une vocation avant tout informative. Une application web permet surtout d'effectuer des actions et de manipuler des données, souvent via un compte personnel.

Normes et validation

▼ Un code HTML valide (qui passe le validateur W3C) est-il automatiquement accessible et compatible avec tous les navigateurs ?

Non. Le validateur détecte les erreurs de conformité HTML, mais ce n'est pas une garantie absolue d'accessibilité ni de compatibilité totale.

La sécurité de base

> ▼ **Qu'est-ce que le phishing ?**
>
> Une technique qui imite un site ou un message légitime (banque, livreur…) pour pousser la victime à communiquer des informations confidentielles.

> ▼ **Copier le contenu d'un autre site (grabbing) est-il toujours illégal ?**
>
> Pas automatiquement : cela dépend du contenu, des droits applicables, de la licence et de l'utilisation qui en est faite. Mais il faut vérifier avant de réutiliser.

8. Exercices

Des exercices **corrigés** pour pratiquer ce que vous avez vu dans cette séance, du plus simple au plus complet. Ils sont **facultatifs et ne sont pas à rendre** : faites-les en autonomie, pendant les séances à distance ou chez vous. Les seuls travaux à déposer dans Teams sont ceux décrits dans les séances 2, 4 et 6. Créez un dossier `exercice-01/` et un fichier par exercice (`ex1.html` , `ex2.html` …).

> ### Essayez d'abord, vérifiez ensuite
>
> Chaque exercice propose un **indice** et une **solution**, cachés : cliquez dessus pour les afficher. Écrivez d'abord votre propre code dans VS Code, testez-le dans le navigateur, et n'ouvrez la solution qu'ensuite pour **comparer**. À l'examen écrit, vous devrez écrire du code sans aide : c'est en cherchant par vous-même que vous vous y préparez.

📝 Envie de vous entraîner encore ? Des exercices courts pour chaque balise vous attendent dans les exercices supplémentaires de la séance 1.

Exercice 1 — Le squelette de mémoire

Partie concernée : le squelette d'une page.

Sans regarder le cours, écrivez une page complète dont l'onglet affiche « Ma première page », avec un titre principal « Bonjour » et un paragraphe de présentation. La page est en français et doit afficher correctement les accents.

▼ Indice

Dans l'ordre : la déclaration du type de document, la racine avec sa langue, puis les deux grandes parties. Le titre de l'onglet et l'encodage vont dans la partie invisible.

▼ Voir la solution

Solution en HTML :

```
<!DOCTYPE html>
<html lang="fr">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Ma première page</title>
</head>

<body>
  <h1>Bonjour</h1>
  <p>Je m'appelle Alex et je découvre le HTML.</p>
</body>

</html>
```

Rendu dans le navigateur :

exercice-01/ex1.html

Bonjour

Je m'appelle Alex et je découvre le HTML.

Pourquoi ? `<!DOCTYPE html>` annonce du HTML5. `lang="fr"` indique la langue aux lecteurs d'écran et aux moteurs de recherche. `<meta charset="UTF-8">` permet d'afficher les accents. Le `<title>` ne se voit que dans l'onglet.

Exercice 2 — Structurer un texte

Partie concernée : titres et paragraphes.

Mettez ce texte en forme avec les bons niveaux de titres et des paragraphes :

```
Bruxelles
La géographie
Bruxelles est située au centre de la Belgique.
Les communes
La région compte dix-neuf communes, dont Evere.
La culture
Bruxelles est connue pour sa Grand-Place.
```

▼ Indice

Un seul `<h1>`. « Les communes » est une sous-partie de « La géographie » : quel niveau, alors ?

▼ Voir la solution

Solution en HTML (contenu du body) :

```
<h1>Bruxelles</h1>

<h2>La géographie</h2>
<p>Bruxelles est située au centre de la Belgique.</p>

<h3>Les communes</h3>
<p>La région compte dix-neuf communes, dont Evere.</p>

<h2>La culture</h2>
<p>Bruxelles est connue pour sa Grand-Place.</p>
```

Rendu dans le navigateur :

exercice-01/ex2.html

Bruxelles

La géographie

Bruxelles est située au centre de la Belgique.

Les communes

La région compte dix-neuf communes, dont Evere.

La culture

Bruxelles est connue pour sa Grand-Place.

Pourquoi ? Un seul `<h1>` pour le sujet de la page, des `<h2>` pour les grandes parties, un `<h3>` pour une sous-partie. On choisit le niveau pour le **sens**, jamais pour la taille.

Exercice 3 — Une image et des liens

Partie concernée : liens et ressources.

Dans `exercice-01/ex3.html` :

1. Affichez l'image `images/photo.png`, avec une description de ce qu'elle montre.
2. Ajoutez un lien vers la page `contact.html`, située dans le même dossier.
3. Ajoutez un lien vers le site du W3C : `https://www.w3.org`.

▼ Indice

L'image utilise `src` et `alt`, les liens `href`. Un lien vers votre site : chemin **relatif**. Vers un autre site : chemin **absolu**.

▼ Voir la solution

Solution en HTML (contenu du body) :

```
<img src="images/photo.png" alt="Un bureau avec un ordinateur portable
ouvert" width="250">

<p><a href="contact.html">Me contacter</a></p>
<p><a href="https://www.w3.org">Le site du W3C</a></p>
```

Rendu dans le navigateur :

exercice-01/ex3.html

Un bureau avec un ordinateur portable ouvert

Me contacter

Le site du W3C

Pourquoi ? `alt` décrit l'image pour ceux qui ne la voient pas. `contact.html` est un chemin relatif (même dossier) ; `https://www.w3.org` est un chemin absolu. Adaptez l'`alt` à **votre** image.

Exercice 4 — Trouvez les 5 erreurs

Parties concernées : le squelette, l'imbrication, les titres.

Cette page s'affiche, mais elle contient **5 erreurs**. Trouvez-les :

```
<html lang="fr">
<head>
  <meta charset="UTF-8">
</head>
<body>
  <title>Mes vacances</title>
  <h1>Mes vacances à la mer</h1>
  <h3>Le premier jour</h3>
  <p>Il faisait <strong><em>très</strong></em> beau.
  <img src="plage.jpg">
</body>
</html>
```

▼ Indice

Regardez la toute première ligne, l'endroit où se trouve le titre de l'onglet, le niveau des titres, l'ordre de fermeture des balises, et ce qui manque à l'image et au paragraphe.

▼ Voir la solution

Code corrigé :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Mes vacances</title>
</head>
<body>
  <h1>Mes vacances à la mer</h1>
  <h2>Le premier jour</h2>
  <p>Il faisait <strong><em>très</em></strong> beau.</p>
  <img src="plage.jpg" alt="Une plage de sable sous un ciel bleu">
</body>
</html>
```

1. `<!DOCTYPE html>` **manquant** en première ligne.

2. **`<title>` dans le `<body>`** : il doit être dans le `<head>`.
3. **Saut de niveau** : après un `<h1>`, on passe à un `<h2>`, pas à un `<h3>`.
4. **Mauvaise imbrication** : la dernière balise ouverte (`<em>`) se ferme en premier. Et le `<p>` n'était pas fermé.
5. **Image sans `alt`**.

Exercice 5 — Entités et commentaires

Parties concernées : les entités, les commentaires.

Écrivez le code qui affiche exactement ces deux lignes dans le navigateur :

```
La balise <p> crée un paragraphe.
Tom & Jerry © 1940
```

Ajoutez au-dessus un commentaire avec votre prénom, qui ne doit **pas** s'afficher.

▼ Indice

Si vous écrivez `<p>` tel quel, le navigateur croit à une vraie balise. Il faut les entités de `<`, `>`, `&` et ©.

▼ Voir la solution

Solution en HTML (contenu du body) :

```
<!-- Exercice 5 — Alex -->
<p>La balise &lt;p&gt; crée un paragraphe.</p>
<p>Tom &amp; Jerry &copy; 1940</p>
```

Rendu dans le navigateur :

exercice-01/ex5.html

La balise <p> crée un paragraphe.

Tom & Jerry © 1940

Pourquoi ? `<` et `>` affichent les chevrons sans créer de balise, `&` affiche le « & » et `©` le ©. Le commentaire `<!-- -->` reste visible dans le code (F12), mais pas dans la page.

Exercice 6 — Écrire les bons chemins

Partie concernée : liens et ressources.

Voici l'arborescence d'un site :

```
mon-site/
├── index.html
├── css/
│   └── style.css
├── images/
│   └── logo.png
└── pages/
    └── contact.html
```

Complétez les chemins relatifs :

1. Dans `index.html`, l'image `logo.png`.
2. Dans `index.html`, un lien vers `contact.html`.
3. Dans `pages/contact.html`, l'image `logo.png`.
4. Dans `pages/contact.html`, la feuille de style.
5. Dans `pages/contact.html`, un lien vers l'accueil.

▼ Indice

> On part toujours du fichier dans lequel on écrit. Depuis le dossier `pages/`, il faut d'abord **remonter** d'un niveau avec `../`.

▼ Voir la solution

Solution de l'exercice 6

Dans le fichier	Cible	Chemin
`index.html`	logo	`images/logo.png`
`index.html`	contact	`pages/contact.html`
`pages/contact.html`	logo	`../images/logo.png`
`pages/contact.html`	feuille de style	`../css/style.css`
`pages/contact.html`	accueil	`../index.html`

Pourquoi ? Depuis la racine, on descend directement dans un dossier. Depuis `pages/`, `../` remonte d'abord à la racine `mon-site/`, puis on redescend.

Les balises HTML vues dans cette séance

Un récapitulatif pour vous y retrouver en révision. Cliquez sur une balise pour revenir à la section qui l'explique.

Glossaire des balises de la séance 1

Balise	Rôle en une ligne	Vue dans
`<!DOCTYPE html>`	Déclare qu'il s'agit d'une page HTML5.	Le squelette
`<html>`	L'élément racine, qui contient toute la page.	Le squelette
`<head>`	Les informations sur la page, invisibles à l'écran.	Le squelette
`<meta>`	Encodage, viewport, description : des métadonnées.	Le squelette
`<title>`	Le titre affiché dans l'onglet du navigateur.	Le squelette
`<link>`	Relie la page à une ressource externe (favicon, CSS).	Le squelette
`<body>`	Tout le contenu visible de la page.	Le squelette
`<p>`	Un paragraphe de texte.	Les balises
` `	Un retour à la ligne à l'intérieur d'un paragraphe.	Les balises
`<img>`	Insère une image (`src` , `alt`).	Les balises
`<hr>`	Marque une rupture thématique dans le contenu.	<hr>
`<strong>` / `<em>`	Importance forte / accent d'insistance (détaillées en séance 2).	L'imbrication
`<!-- -->`	Un commentaire, ignoré par le navigateur.	Les commentaires
`<h1>` à `<h6>`	Les six niveaux de titres, pour le sens, pas la taille.	Titres et paragraphes
`<a>`	Un lien hypertexte (`href`) détaillé dans séance ultérieure.	Liens et ressources
`&` , `<` …	Les entités, pour afficher des caractères réservés.	Les entités HTML

Découverte des outils

Avant de nous lancer, un rapide tour des outils que vous utiliserez tout au long du cours :

- **Le navigateur** : Chrome, Firefox et Edge sont les plus utilisés. Installez-en au moins deux pour pouvoir tester vos pages dans plusieurs navigateurs.
- **VS Code** : l'éditeur de code que nous utiliserons, avec l'extension **Live Server** pour prévisualiser une page HTML directement dans le navigateur, avec rafraîchissement automatique.
- **Teams** : la plateforme où les fichiers du cours sont déposés chaque semaine, et où vous déposerez vos propres travaux.

L'installation détaillée de ces outils se fait à la séance 2 : voir le guide d'installation.

Le travail pour la séance 2

La séance 2 se fait **à distance**. Vous y installerez votre environnement de travail et vous écrirez votre première page complète.

Le détail est sur la page de la séance 2. Commencez par le guide d'installation.

Sources et ressources

- Syntaxe de base du HTML — MDN
- Métadonnées des pages web — MDN
- Titres et paragraphes — MDN
- Validateur HTML du W3C

Les images utilisées dans ce cours proviennent de diverses sources, notamment :

- https://www.sanzaru.be/menu/
- https://code-garage.com/blog/quelle-est-la-difference-entre-internet-et-le-web
- https://disenowebakus.net/en/world-wide-web
- https://www.britannica.com/biography/Tim-Berners-Lee
- https://www.w3.org/Consortium/mission
- https://techdifferences.com/difference-between-web-page-and-website.html
- https://www.malekal.com/le-modele-ou-architecture-client-serveur/
- http://nsinfo.yo.fr/nsi_prem_url.html

- https://openclassrooms.com/en/courses/1603881-creez-votre-site-web-avec-html5-et-css3/8061261-creez-votre-premiere-page-web-en-html
- https://www.idcalg.com/?normes-W3C-conception-site-web#r11
- https://www.magnific.com/fr/photos-vecteurs-libre/vacances

EAFC Evere — UE Web : principes de base — 40 périodes. Ce site est écrit à la main en HTML et CSS : faites `F12` pour en inspecter le code.

Certains contenus s'appuient sur la documentation de MDN Web Docs et d'autres ressources trouvées en ligne (textes, images), ainsi que sur une assistance de l'IA (Claude, Anthropic) pour la reformulation de certains passages.Certaines images ont été générées par l'IA (ChatGPT, OpenAI).